Vocomatcher: Interactive Web-App for Matching Animal Vocalisations in Passive Acoustic Monitoring Surveys

Antonios Ntalamagkas



University of
St Andrews

This dissertation is submitted in partial fulfilment for the degree of

MSc Data-Intensive Analysis

at the University of St Andrews

August 13, 2024

Abstract

Matching similar animal calls captured by multiple microphones in terrestrial acoustic surveys in order to identify individual callers is a key step in bioacoustics research. Pinpointing individuals helps answer important ecological questions, including estimating the population size within a study area, tracking the spatial distribution of individuals over time, and assessing the effects of environmental changes—such as habitat degradation or climate change—on vocal behaviour and movement patterns. These insights are crucial for driving change in environmental policy and coordinating conservation efforts, especially for endangered species.

Currently, manually matching animal vocalisations by ear and eye is the primary method used for identifying individuals. This task requires extensive training, focus, and patience to perform accurately. Hence, there is an urgent need for human-assistive software that can expedite the call matching and deduplication process.

In this dissertation, we document the development of Vocomatcher, a web application that assists ecologists with grouping similar animal calls via an interactive, user-friendly interface. Vocomatcher employs two main visualisation techniques to facilitate call matching: a map displaying the positions of detectors within the study area, allowing users to compare the angles from which calls were detected, and an interface that enables the comparison of calls based on their spectrogram images.

To our knowledge, no other software tool offers the same functionality. Vocomatcher is free and open-source. The source code is available at github.com/jaytohe/ecostats-rshiny.

Declaration

I hereby certify that this dissertation, which is approximately 13,949 words in length, has been composed by me, that it is the record of work carried out by me and that it has not been submitted in any previous application for a degree.

This project was conducted by me at the University of St Andrews from May 2024 to August 2024 towards fulfilment of the requirements of the University of St Andrews for the degree of MSc Data-Intensive Analysis under the supervision of Dr Areti Manataki and Dr Hannah Worthington.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.



ANTONIOS NTALAMAGKAS

August 13, 2024

CONTENTS

Contents **i**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Contributions . . . 3
1.3 Project Aims . . . 3
1.3.1 Primary Objectives . . . 3
1.3.2 Secondary Objectives . . . 4
1.3.3 Tertiary Objectives . . . 4
1.4 Dissertation Overview . . . 4

2 Context Survey **6**
2.1 Wildlife Survey Pipeline . . . 7
2.1.1 Monitoring Methods . . . 7
2.1.2 Classification . . . 10
2.1.3 Localisation and Density Estimation . . . 13
2.2 Related Work . . . 16
2.2.1 Koe . . . 16
2.2.2 Unsupervised Classification . . . 20
2.2.3 The "ecostats-rshiny" App . . . 24
2.3 Literature Gap . . . 31
2.4 Background Information . . . 31
2.4.1 Shiny Framework and Reactive Programming . . . 31

3 Methodology **35**
3.1 Waterfall Methodology . . . 35
3.2 Why Was the Waterfall Methodology Chosen? . . . 36

4 Requirements Specification **37**
4.1 Approach to Requirement Collection . . . 37
4.1.1 Contextual Inquiry . . . 37

4.1.2 Independent Evaluation of "ecostats-rshiny" 38
4.1.3 Requirements Classification and Prioritisation 38
4.2 Requirements Gathered 39

5 Design **45**
5.1 Wizard UI Pattern 45
5.2 UI Mock-ups 46
5.2.1 "Import CSV" Screen 46
5.2.2 "Date Select" Screen 47
5.2.3 "Match Calls" Screen 48
5.2.4 "Export Calls" Screen 51

6 Vocomatcher - Software Implementation **53**
6.1 Final Look 53
6.2 Implementation Details 55
6.2.1 Input Data Format 56
6.3 Challenges Encountered 56
6.3.1 Displaying Images Inside a DataTable 56

7 System Testing **59**
7.1 Unit Testing 59
7.2 Integration Testing 59
7.3 Call Removal Bug 60

8 Conclusion **62**
8.1 Project Summary 62
8.2 Objectives Review 64
8.3 Limitations 65
8.4 Future Work 66

References **67**

CHAPTER ONE

INTRODUCTION

This chapter briefly explains the motivation behind developing Vocomatcher, presents the project's objectives, and outlines the content of the following chapters.

1.1 Motivation

Researchers at the University of St Andrews' Centre for Research into Ecological and Environmental Modelling (CREEM) have deployed recording units in an area in Laos to study gibbon monkeys. Gibbons produce loud and distinctive vocalisations, making them ideal subjects for passive acoustic monitoring using microphones. The ultimate aim of their research is to estimate the population density of gibbons, expressed as the number of gibbons per unit area, using a statistical model called Spatially Explicit Capture Recapture (SECR), discussed in section 2.1.3.1.

To achieve this, researchers must be able to:

1. Uniquely identify individual gibbons based on their vocalisations.

2. Estimate the angle (bearing) from which each call was heard on a microphone.

The former is necessary in order to sample the number of individuals present within the study area, and is required to apply SECR models, while the latter provides additional spatial information that can help pinpoint the locations of calling gibbons more accurately. For more details, see section 2.1.3.

However, uniquely identifying a vocalising individual is challenging given that a single gibbon call is recorded by multiple microphones. Furthermore, it can be difficult to decipher whether there are multiple individuals or a single individual vocalising repeatedly [1].

Researchers must manually review the recorded data, listen to the audio, and visualise it to identify and deduplicate gibbon calls. This also involves comparing:

(a) The time each call was heard

(b) Which microphone recorded each call

(c) The estimated bearing of each call

Extensive training is required to perform this task accurately. This is because the repetitive nature of the work requires prolonged periods of intense concentration. Maintaining focus is crucial to avoid mislabelling calls, i.e. assigning a call to individual x when it truly belongs to individual y. All in all, it's a tough job that requires a lot of patience and skill to get right.

Call matching is not the only task where manual human expert-based analysis is the primary method. In fact, during our literature review, we discovered a recent survey paper from 2019 indicating that **58% of all terrestrial acoustic studies conducted up until then relied on manual acoustic analysis** [2].

Therefore, it is evident that a solution be developed to expedite the process of matching and deduplicating similar calls recorded by multiple microphones.

Researchers at CREEM do not want a machine learning-based solution for this task. Instead, they would like a human-in-the-loop program which assists in call matching, keeping the whole process explainable and transparent.

In their search for a suitable tool, they identified an existing application on GitHub named "ecostats-rshiny", presented in section 2.2.3, which visualises the estimated bearings on a map to assist with call matching. However, this application is currently unfinished, suffers from bugs, crashes occasionally, and the researchers are not convinced that its bearings visualisation is sufficient for effective call matching.

1.2 Contributions

This is where Vocomatcher comes into play. Vocomatcher is a complete re-write of "ecostats-rshiny," designed to address its limitations while retaining its current bearings visualisation map. In addition, Vocomatcher introduces new functionalities and usability enhancements as requested by our client, researchers at CREEM. Some of the new functionalities include:

- A **revamped user interface** for matching calls,
- The ability to **compare calls based on their spectrograms**[1], not just their bearings,
- Exporting matched calls into a file format which is **directly compatible** with software used for SECR analysis

1.3 Project Aims

Below, we outline the objectives we sought to achieve as part of this project:

1.3.1 Primary Objectives

1. Sketch design mock-ups and revise them based on feedback from the client.
2. Develop the new app using a modular approach to improve code debugging and maintainability.
3. Integrate useful features from "ecostats-rshiny" into Vocomatcher.
4. Implement new functionalities as requested by the client, including the ability to compare animal calls based on spectrograms.
5. Properly test the app's components to ensure stability and to minimise the risk of crashing.

[1] Spectrograms are discussed in section 2.1.2.1.

1.3.2 Secondary Objectives

1. Design and implement an algorithm that will try to guess which calls are the same based on time of arrival, sex of the animal and location of the recorders.

2. Develop a user interface that allows users to accept or reject call matchings proposed by the algorithm.

1.3.3 Tertiary Objectives

1. Implement an autosave functionality to allow users to restore the app's state if it closes or crashes.

2. Package the app using containerisation software, such as Docker, for easy installation and deployment.

Note: The project's objectives have been heavily modified since the DOER submission. Above, we presented the final iteration of the revised objectives.

1.4 Dissertation Overview

Chapter 2 provides the necessary background information on wildlife surveys, presents related work to Vocomatcher, including the app it is based on, "ecostats-rshiny," and discusses the Shiny web app framework used to develop Vocomatcher.

Chapter 3 outlines the software engineering methodology followed in this project.

Chapter 4 details how the software requirements were elicited and presents the gathered requirements.

Chapter 5 details the UI pattern we decided to follow and the rationale behind our choice. It also presents the UI mock-ups developed through continuous collaboration with the client, including a new design for the call-matching user interface. We provide a comprehensive explanation of how this screen is intended to be used.

Chapter 6 showcases Vocomatcher's final look and usage, provides details on its implementation, and discusses interesting challenges faced during development and their solutions.

Chapter 7 explains how the app was evaluated, including how complex components were tested.

In Chapter 8 we summarise the insights gained from this dissertation, reiterate the objectives tackled by Vocomatcher, and discuss its current limitations and potential future work.

CHAPTER TWO

CONTEXT SURVEY

This chapter in divided into four sections:

- Section 2.1 describes how wildlife surveys are conducted using microphones compared to cameras. The different methods of observing, detecting and classifying animals are discussed, and the mathematical models used for estimating the number and location of individuals are presented.
- In Section 2.2, existing methods and software tools for classifying and matching vocalizations are reviewed, including the "ecostats-rshiny" app. Moreover, the advantages and limitations of each method are discussed.
- Section 2.3 explains the literature gap we have identified.
- Section 2.4.1 introduces the reader to the reactive programming paradigm and the Shiny web-app framework, which is used to develop the app presented in this dissertation, Vocomatcher.

2.1 Wildlife Survey Pipeline

A wildlife survey is a systematic framework of collecting and analysing information on animal populations and their habitat. Wildlife surveys differ according to methods used for data collection and their main research objective. For example, one popular objective is calculating the number of animals per unit area, i.e. the animal density. Another example is estimating the mortality rate of a given animal population.

Wildlife surveys are essential for gathering data to conserve endangered species and their habitats. Most Gibbon species, for example, are either endangered or critically endangered [3], with the Hainan gibbon being the rarest primate in the world [4]. These surveys also play a crucial role in environmental policymaking and in evaluating the impact of human activities. For instance, they can be used to study how deforestation affects the behaviour and movement patterns of Gibbons.

In this dissertation, we are focusing on the estimation of animal density. This is achieved via a pipeline comprising four steps, as shown on Figure 2.1.

In the following subsections, each step of the aforementioned pipeline is discussed.

2.1.1 Monitoring Methods

There exist two main classes of monitoring: Invasive monitoring and passive monitoring. In the former, the researcher physically traps the animal and marks it using colour, a tag, collar or implant. This allows them to track the animal and its whereabouts and uniquely identify it. However, physically trapping an animal is not always feasible. Some animals may be small, elusive, nocturnal or hidden underground. Furthermore, capturing an animal raises ethical and welfare concerns, and it interferes with their natural pattern/behaviour. Therefore, passive techniques that do not require physical interaction are preferred.

Below, we review two passive monitoring methods: Camera trapping, and Passive Acoustic Monitoring.

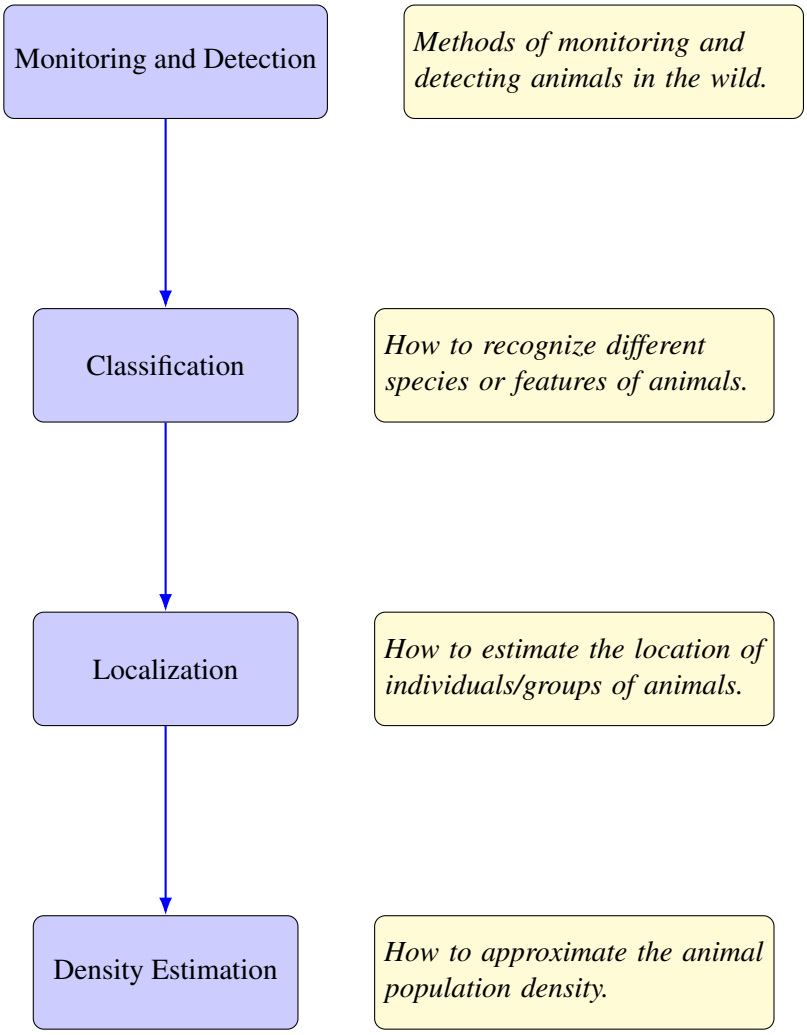


Figure 2.1: The steps needed to calculate the density of an animal population via a wildlife survey.

2.1.1.1 Camera Trapping

Camera trapping entails deploying multiple cameras at the study site such that photos can be automatically captured in the presence of an animal. These cameras are typically equipped with a motion sensor, a flash and/or an infrared sensor such that photos can be taken during nighttime and without disturbing the animal. Compared to human observers, camera traps can operate continuously for extended period of times (months or even years) and they can be deployed in remote locations which are hard to reach

by researchers. This means that even the rarest events, such as the passage of an apex predator through an area, can be captured and studied [5, p.2].

Camera traps can be used for population density estimation by identifying individuals from the photos taken, as we will discuss shortly, and noting which camera and at what time captured each individual. Moreover, by incorporating GPS telemetry data, animal movement and space occupancy can be modelled, as in [6].

However, camera traps are not a perfect solution for wildlife surveying. First, motion-activated camera traps can be overly sensitive to small movements caused by wind or plants, leading to numerous blank pictures. These unnecessary images quickly fill up the camera's flash memory and drain its battery. Fortunately, there is ongoing research on embedding machine learning models in the cameras' software to automatically filter out blank images, see [7]. Furthermore, infrared sensor based cameras can miss detections under hot weather due to a lower differential between the background surface temperature and the target animal [8]. This implies that the animal must be closer to the camera to trigger it.

2.1.1.2 Passive Acoustic Monitoring

Some species may be hard to see but easy to hear. Indeed, many animals produce distinct sounds that can be detected acoustically at greater distances compared to visual methods like camera traps. This is particularly true underwater, where sound travels faster than in air. For example, marine mammals communicate by slapping their body parts, while odontocetes, such as beluga whales, use echolocation to navigate dark waters or avoid obstacles [9].

Passive Acoustic Monitoring (PAM) entails using autonomous recording units (e.g. microphones, hydrophones, ultrasound detectors) to record audio data in the field and detect animal vocalisations [10]. PAM has long been used for the study of marine mammals and fish, but it has only recently gained traction as a means of terrestrial habitat monitoring [1, Section 3.2 p. 964][2].

Advantages of PAM

1. Animal sounds are highly informative. Vocalisations encode information about

abundance, position, body size and interactions in space and time [10, Sec. 2].

2. Sound propagates in multiple directions, through physical objects, and over long distances, making it easier to hear animals rather than see them.

3. Multiple species can be studied at the same time using the same equipment. Many devices are capable of recording human audible range as well as infra- and ultrasound [1, Fig. 1].

4. Long-term PAM data allows the examination of temporal variations in the vocalisations of the target species. Valuable ecological questions such as the role of the lunar cycle in a species natural history can be answered [1, Sec 2.2].

Limitations of PAM

1. Networks of audio recorders produce a lot of data, which can be challenging to store and process. Compared to camera traps, microphones record data continuously, which requires high storage capacity.

2. Manual processing of captured audio recordings to detect and classify species is the norm. [2] found that 58% of terrestrial PAM studies performed manual acoustic analysis. This is both laborious and time-consuming for researchers.

3. Identifying individuals from vocalisations is difficult; it is challenging to decipher whether there are multiple individuals versus a single individual vocalising repeatedly.

4. A species that was not detected does not mean it is absent; it may simply have been too quiet to be detected [1].

2.1.2 Classification

Classification entails recognising different species or certain characteristics of a given species e.g. sex, weight, depending on the design of the wildlife survey.

In camera trap-based studies, such features are extracted from visual cues present in the images captured. For instance, to determine the sex, the ecologist may inspect the colour, size of an animal, or they may inspect the shape of their genitalia. In the case of adult



Figure 2.2: Two southern white-cheeked gibbons. The left one is a female, evident from her light fur. The right one is a male, evident from his black fur. F.Spangenberg, CC BY-SA 3.0 <https://creativecommons.org/licenses/by-sa/3.0>

white-cheeked gibbons, sex can be distinguished by fur colour: adult females always have tan fur, while adult males have black fur [11], as seen in Figure 2.2.

In recent years, researchers have increasingly employed deep neural networks, particularly image classification models, to detect target species in captured images [12, 13, 14].

2.1.2.1 Spectrogram

In PAM studies, researchers typically visualise the captured audio data to perform detection and classification. In particular, spectrograms are used.

A spectrogram is a 2D visual representation of the frequencies that comprise a signal and how those vary over time. The vertical axis represents frequency, with higher-pitched sounds corresponding to higher frequencies. Time progresses from left to right on the horizontal axis. The amplitude of a particular frequency at a given timestamp is encoded by colour saturation. The brighter the colour, the larger the amplitude and consequently the louder the sound is. A spectrogram of a female Gibbon call is shown on Figure 2.3. The female Gibbon vocalises the loudest from 0 to 5s at frequencies 500-1200Hz, denoted by the bright red colour. Additionally, the duration of individual sounds within the call decreases over time.

Mated male and female gibbons are known to vocalise consecutively in what are called *duets* [17]. An example of such a duet is illustrated in Figure 2.4. Observe that the

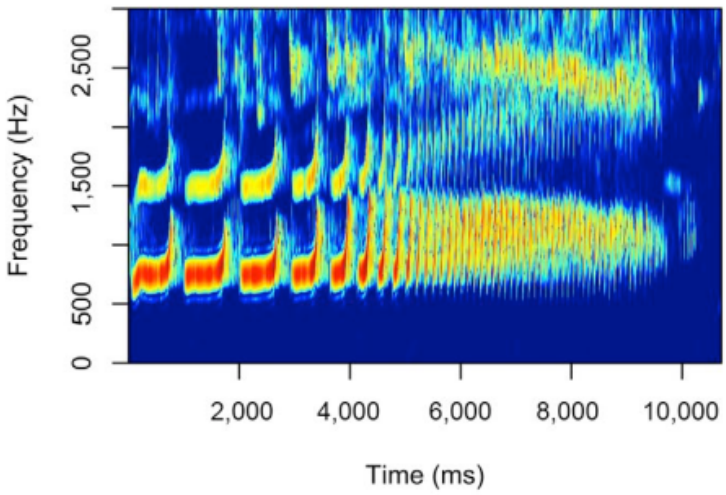


Figure 2.3: Spectrogram of a female Gibbon call. Credit: [15, Fig. 1], Re-used with permission from "John Wiley and Sons"

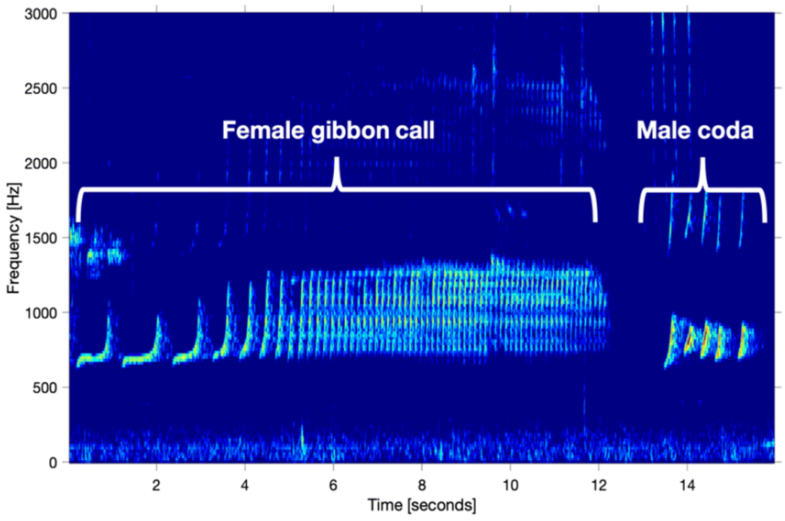


Figure 2.4: Representative spectrogram of the duet contributions of a female and male gibbon. The duet begins with a long female vocalization and ends with a short reply from the male. Credit: [16, Fig. 2], Re-used with permission from "Springer Nature"

female's call lasts longer (~12 seconds) than the male's (~2 seconds) and has a distinctly different appearance. Moreover, observe the similarity in the female calls shown in Figure 2.3, Figure 2.4. Researchers use these visual cues and others extracted from spectrograms to detect gibbon vocalisations and classify them by sex.

2.1.3 Localisation and Density Estimation

Localisation entails determining the spatial position of animals' activity centres, i.e. where they are more likely to be found. While localisation is not necessary for density estimation, it is essential for having *accurate and robust* density estimates. Before discussing why that is the case, let us present the different methods that can be used for animal density estimation.

First, why are mathematical models needed to estimate the population size? It is challenging to detect all individuals in a study area, regardless of the monitoring method used. Thus, inferences must be scaled up to the population size based on a limited sample of questionable representation via statistical assumptions.

One way to measure the population size in a given area is to first determine whether an animal has been captured before or not. This is the approach taken by **capture-recapture (CR) models**. The animal is captured, marked and released. If it is seen again, then the mark makes it apparent that this is not a new sighting. Records of capture and recapture events for individual animals over multiple sampling occasions, so-called capture histories, are created, which are input to the CR models to estimate the population size.

However, conventional CR models make assumptions which do not hold in practice. First, it is assumed that there exists **no spatial variation** in the **capture probability** [18]. In reality, animals closer to traps are more likely to be caught. Additionally, the location of a trap within the study area also affects the capture probability. Second, population size (N) estimates derived from CR models are difficult to convert into density estimates. The main issue is the lack of a precise method to determine the effective trapping area (ETA) covered by the sampling [19]. ETA is assumed to be the area defined by the traps + a distance W from the traps, where methods for estimating W are ad-hoc. For example, $W = 0.5 \times MMMD$ where $MMDM$ is half of the maximum distance moved between

captures, averaged over all individuals [20, p. 231]. All in all, imprecise estimation of N and ETA leads to unreliable density estimates.

2.1.3.1 Spatially-explicit capture recapture model

Spatially-explicit capture recapture (SECR) models [18, 21] aim to tackle the two aforementioned issues i.e. unmodelled heterogeneity in detected animals due to spatial variation, and unreliable density estimation, by **explicitly requiring the collection of spatial information for captured animals**. At the very least, the location of the traps in which animals are detected must be supplied to the model. Furthermore, SECR models eliminate the need for estimating the ETA by directly computing the population density.

SECR Input Data

1) Spatio-temporal capture histories must be collected for i individuals, detected on j traps on k occasions. It is assumed that traps are checked at regular intervals and the time between each trap check is an occasion [18]. An example of capture data is shown on Table 2.1. Individual #1 is caught on trap #1 on two occasions, while individual #2 is caught on trap #4 the first time and trap #3 the second time.

ID	TrapID	Occasion
1	1	1
2	4	1
3	2	1
1	1	2
2	3	2

Table 2.1: Example SECR capture history data. There are 3 individuals, 4 traps and 2 occasions.

2) The location of each trap must be supplied to the model in the form of Cartesian coordinates, as shown on Table 2.2.

SECR Components

SECR models consist of two key components:

- an observation process model, which encapsulates the probability of detecting an animal at the trap locations. This probability depends on the individual's spatial

TrapID	x	y
1	0	0
2	72.32	9.1
3	176.22	9.1
4	182.88	0

Table 2.2: Trap placement data for the example data shown in Table 2.1.

location and typically decreases with increasing distance from the individual's activity centre to the detector.

- a spatial point process model, which describes how the number and distribution of animals' activity centres in space arise.

SECR and Camera Trapping

As mentioned previously, **individual identification** is required to create capture histories for capture-recapture models and SECR. In camera-trap studies, researchers rely on species-specific natural marks present in captured images to identify individuals. For instance, in tigers and jaguars the pelt pattern is unique and used to identify each individual [22, 23]. When individuals lack unique natural markings, researchers must rely on characteristics that may not be present in every individual. For example, in [24], a combination of sex, tail length and tail kinks, "hanging" bellies of lactating females, body slimness, and torn ears were used to identify ocelots.

Moreover, in order to calculate the occasion, timestamps from the metadata of the captured images are used.

SECR and Acoustic Detectors

In terrestrial capture-recapture acoustic surveys, multiple recording units are strategically placed around the survey area to ensure that animal calls are detected by several units and each microphone within those units. By recording the time of arrival of each call and using specialised software to estimate the detection angle from each unit, triangulation is used to determine the approximate location of the calling animal based on data from all microphones which detected the call.

In SECR literature, acoustic detectors are known as *proximity* sensors. Unlike physical and camera traps, a single individual can be "captured" by multiple proximity detectors during a single sampling occasion. This allows for density estimation based on just one sampling occasion, as opposed to needing multiple ones [21].

To determine which sounds detected on multiple microphones belong to the same vocalising source, researchers typically manually review recordings by listening to them, visualising spectrograms, and comparing times of arrival and bearings. Capture histories are then created by matching each distinct vocalisation to the recorders that detected it.

In the next section, we discuss some methods to potentially speed up the manual process of matching calls.

2.2 Related Work

In this section, we review three existing methods which address the challenge of manually disentangling and deduplicating calls to identify individuals. First, we present Koe, a semi-automated solution for acoustic unit classification, particularly useful for birdsong analysis. Next, we discuss a paper that introduces machine-learning-based methods for the acoustic classification of Gibbon vocalisations. Finally, we showcase the "ecostats-rshiny" app, which we have forked, enhanced, and used as the foundation for developing Vocomatcher.

2.2.1 Koe

Koe [25] is an application designed for classifying and analysing animal vocalisations. In particular, the authors focus on animals who have flexible vocalisations, and it is difficult to distinguish different calls/individuals based on whole songs. A prime example of this are bird songs, where one bird sings multiple songs and factors such as sex and fitness of the individual influence their vocal characteristics [26]. Hence, to study these songs, they must be broken down into segments called *syllables*. These syllables are then classified into specific categories. After classification, researchers can identify patterns in birds' vocal behaviour and assess similarities between vocalisations.

Motivation behind Koe

Classifying acoustic units is challenging, and it involves visualising spectrograms, listening to audio playback, and comparing units in order to manually label them. This process is laborious and time-consuming.

Moreover, while automated methods for classification have been explored, these are mostly ad-hoc, not transparent and not generalisable. In fact, the second paper presented in this chapter, in section 2.2.2, is one such method. Finally, the authors of Koe wanted to create a tool that can be used to classify vast amounts of acoustic units in bulk, while existing software only supported processing a few files at a time.

Koe's functionality

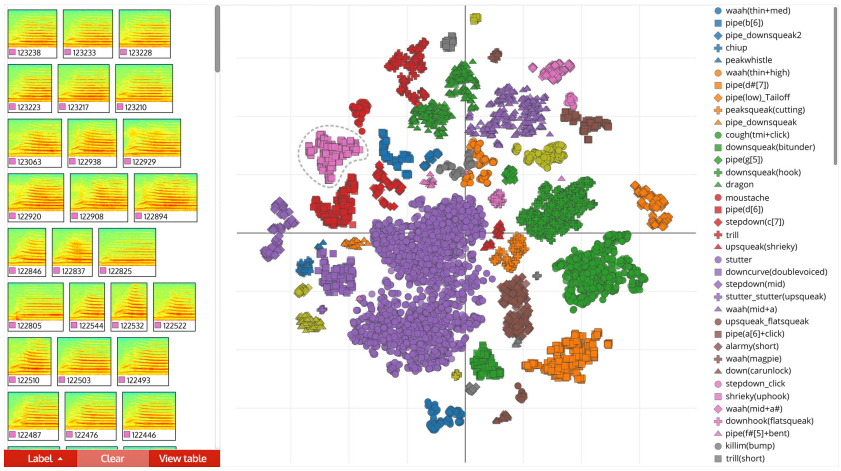


Figure 2.5: Koe's interactive ordination plot, visualizing the syllables of bellbirds songs. The user can lasso-select groups of points, view the relevant spectrograms on the left-hand side, listen to the audio of each syllable and label the acoustic units in bulk. Clicking on "View Table" opens up the unit table shown on fig. 2.6. Credit : [25, Fig. 3]; Re-used with permission from "John Wiley and Sons"

Koe provides a web-based user interface which offers the ability to upload, play back, visualise, filter and segment audio data, compare acoustic units via similarity scores and bulk-label them via interactive plots, all in one program.

The user uploads raw recording data and then finds the songs within the recordings by listening to them or visualising the spectrograms. Then, the user segments the recordings



Figure 2.6: Koe's unit table view. Koe's unit table is a tool for organising and analysing audio units. It displays information about each unit, including a spectrogram that can be enlarged for closer inspection. Users can listen to the audio by clicking on the spectrogram. The table allows units to be sorted and filtered based on different characteristics. Specifically, sorting by similarity index helps users quickly identify units with similar acoustic properties for rapid labelling. Credit: [25, Fig. 4]; Re-used with permission from "John Wiley and Sons"

to the songs by picking the start and end timestamp for each song. Afterwards, the user segments the songs into acoustic units via the same method. After that, various spectral/temporal features are extracted from the acoustic units[1] and dimensionality reduction [27] is applied on them. Finally, a similarity score is calculated between each pair of acoustic units via agglomerative hierarchical clustering [28].

By applying dimensionality reduction, Koe can effectively visualise all the acoustic units into a single 2D interactive plot, as shown in Figure 2.5. In said plot, the user lasso-selects groups of points to play back their audio and view their spectrograms. The selected acoustic units can be labelled directly on the plot or opened in a table to compare them more thoroughly.

The respective table for the selected acoustic units is shown in Figure 2.6. Each row in the table corresponds to a unit, and it contains the spectrogram, ID, the date it was captured and other information. The table can be filtered/sorted by any column. Clicking on a spectrogram, enlarges it and plays the audio. Finally, sorting by the similarity index,

[1] In Koe, the user can select which unit features to extract. Click here to see the available choices of features.

ranks the units based on their acoustic similarity. This makes labelling large data batches easier and faster.

Once the data has been labelled, the sequences of acoustic units can be analysed in the app, or they may be exported to a CSV file.

Technical Details

Koe is a web-based application written in Django, a Python-based web framework. It is accessible at the following URL : koe.io.ac.nz. The user first needs to create an account to access the service. Registering an account and using the website is free-of-charge at the time of writing this. All user-uploaded data is stored in a MySQL database. Uploaded data can be shared across users, allowing them to collaborate.

Below, we outline some benefits and drawbacks of Koe we have identified:

Benefits of Koe

- ☑ Web-based app
  - ☑ Runs anywhere a web browser can
  - ☑ No setup/installation is needed
  - ☑ Can run on multiple browser windows at once ⇒ useful for having different app screens open simultaneously
- ☑ It is free and open source: github.com/fzyukio/koe
- ☑ Supports multiple users and collaboration between users
- ☑ Provides a unified interface for working with acoustic data
  - ☑ Supports the most common bio-acoustics processing tasks e.g. playback, spectrogram visualisation, audio segmentation, classification
- ☑ Offers a semi-automated solution for acoustic unit classification
  - ☑ Combines human expert judgement and audiovisual perception with numerical features extracted from the data to classify acoustic units via interactive plots

☑ By calculating a similarity score, it provides a hint as to which acoustic units may be the same which facilitates rapid data labelling in bulk

Limitations of Koe

Given that Koe is a web-based app that does all the heavy processing on the server-side, **uploading large files and audio data can be time-consuming**. An alternative would be to run Koe locally. However, this approach would sacrifice the convenience and scalability that Koe's web-based architecture provides, as well as the ability to access and collaborate on data from anywhere. Moreover, Koe relies on a machine learning library to fully function[2] which requires a powerful CPU or GPU.

Furthermore, the authors state that Koe has been **primarily designed for acoustic surveys which require acoustic unit classification,** e.g. birdsong analysis. In the context of density estimation of Gibbons, while the similarity index and the interactive ordination plot can be useful to group calls belonging to the same individual and to label their sex, there is **no visual aid in localising said calls**. That is because Koe accepts only recording data as input and not microphone location coordinates.

2.2.2 Unsupervised Classification

Supervised vs Unsupervised Learning

An alternative approach to matching calls is to train an algorithm to predict the individual a call belongs to. In the machine learning landscape, there are two main ways to accomplish this: supervised learning and unsupervised learning. The former is applicable when **labelled** datasets exist. In the acoustic survey context, said dataset may consist of columns of spectral/temporal features extracted from the raw recording data, as discussed in section 2.2.1, and a column indicating the identity of an individual. The supervised algorithm learns to predict the identity of a call based on the features. Supervised techniques for vocal recognition of animals have been explored thoroughly both terrestrially ([29] [30] [31] [32]) and underwater ([33], [34], [35]). However, they all have the same pitfall: **individual identity must be known beforehand**, which is often not the case for acoustic data collected over extended periods. In contrast, unsupervised

[2]Koe requires the Keras machine learning library as shown here.

classification algorithms are used to group similar calls together, without requiring labelled data.

Unsupervised Acoustic Classification of Female Gibbons

This section focuses on unsupervised acoustic classification of individual Northern gray gibbon (Hylobates funereus) females, as explored in [15]. The authors utilise a labelled dataset of female calls collected from five sites in Malaysian Borneo to evaluate the efficacy of unsupervised clustering techniques in distinguishing individual females. In particular, three different clustering algorithms are used: K-medoids [36], Gaussian mixture models [37], and Affinity Propagation [38].

For each algorithm, they deduce how well every data point fits into the cluster it gets assigned to compared to other clusters, by calculating the silhouette score [39, p. 245-247] In addition, since the dataset is labelled, the unsupervised cluster results are compared with the known female identity via the normalized mutual information (NMI) score [40] and by calculating the number of calls that are correctly classified. This is done by assuming that all calls in a given cluster belong to the female with the highest number of calls in said cluster.

Finally, to compare the algorithms, they randomly sample with replacement 10, 20, 30, 40 and 50 females from the dataset over 100 iterations and calculate the aforementioned silhouette, NMI and percent correct metrics. Furthermore, to test how the clustering results vary by study site, they randomly choose 80% of calls per site and compute the same metrics.

Clustering Results

Figure 2.7 includes a screenshot of [15, Table 2] which neatly summarises the results of running each clustering algorithm on 10, 20, 30, 40 and 50 Gibbon females randomly sampled with replacement for 100 iterations. The silhouette score, NMI and percentage of correct classifications are measured. Notice that the mean percentage of correctly classified females of Affinity is consistently higher than that of Gaussian and K-medoids for all sampled number of females, while the standard deviation of the correct percentage is lower compared to the other two algorithms. Moreover, observe that the mean number of clusters fitted via affinity propagation is closer to the true number of females and has a

lower standard deviation than K-medoids and Gaussian.

Number of females	Clustering method	Number of calls (range)	Number of clusters ($M \pm SD$)	Percent correct ($M \pm SD$)	Silhouette ($M \pm SD$)	NMI ($M \pm SD$)
10	Affinity	102–238	12.49 ± 2.02	75.91 ± 12.76	0.33 ± 0.05	0.72 ± 0.14
	Gaussian	100–253	15.52 ± 8.24	70.98 ± 20.46	0.28 ± 0.07	0.73 ± 0.24
	K-medoids	106–274	7.18 ± 3.92	53.24 ± 18.25	0.37 ± 0.04	0.55 ± 0.28
20	Affinity	224–445	22.88 ± 2.31	72.55 ± 9.31	0.28 ± 0.02	0.59 ± 0.14
	Gaussian	267–436	25.34 ± 9.3	50.78 ± 9.78	0.18 ± 0.04	0.45 ± 0.12
	K-medoids	240–444	9.10 ± 8.61	34.59 ± 15.25	0.32 ± 0.03	0.33 ± 0.31
30	Affinity	403–601	32.31 ± 2.53	68.59 ± 5.44	0.25 ± 0.02	0.43 ± 0.09
	Gaussian	425–588	31.42 ± 9.47	50.36 ± 7.38	0.17 ± 0.02	0.43 ± 0.11
	K-medoids	405–602	10.89 ± 11.87	26.65 ± 12.13	0.28 ± 0.03	0.25 ± 0.27
40	Affinity	592–738	40.91 ± 2.29	66.80 ± 3.45	0.23 ± 0.01	0.34 ± 0.06
	Gaussian	585–732	36.07 ± 7.38	49.39 ± 6.14	0.16 ± 0.02	0.39 ± 0.09
	K-medoids	581–737	9.50 ± 12.96	19.51 ± 8.74	0.26 ± 0.02	0.17 ± 0.23
50	Affinity	784–860	50.15 ± 1.29	64.83 ± 2.05	0.21 ± 0.01	0.27 ± 0.03
	Gaussian	793–856	38.7 ± 5.14	49.17 ± 3.86	0.15 ± 0.01	0.39 ± 0.06
	K-medoids	782–857	5.72 ± 9.38	16.26 ± 4.26	0.25 ± 0.01	0.08 ± 0.13

Figure 2.7: Comparison of three unsupervised classification algorithms (Affinity propagation, K-medoids and Gaussian mixture model) applied to 10, 20, 30, 40 and 50 females randomly sampled with replacement. The number of clusters estimated by each algorithm along with the percentage of correct classifications, the silhouette score and the normalised mutual information index (NMI) are computed. A higher NMI or Silhouette score is better. Credit: [15, Table 2], Re-used with permission from "John Wiley and Sons"

Furthermore, Figure 2.8 shows a screenshot of [15, Fig 4] which depicts the performance of the three clustering algorithms across the five different study sites. For each site, 80% of the calls are randomly sampled over 100 iterations. Notice that when comparing the number of clusters to the true number of individuals, affinity propagation has considerably lower deviation from the actual number of females, evident from the small interquartile range (shown in yellow on bottom graphic). Additionally, the median percentage of correctly classified females of affinity propagation is consistently higher across all sites compared to the other two algorithms.

From these experiments, the authors conclude that affinity propagation may be useful for estimating the number of vocalising individuals in PAM studies, albeit with some error.

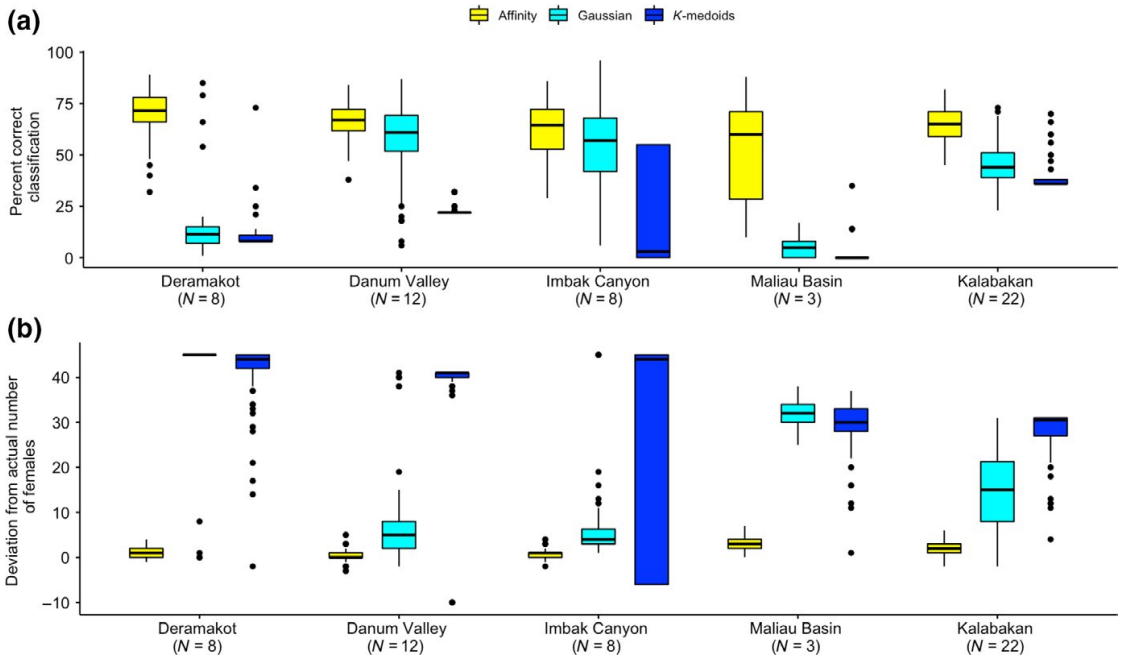


Figure 2.8: Comparison of three unsupervised classification algorithms (Affinity propagation, K-medoids and Gaussian mixture model) applied to data from five different Gibbon study sites. For each site, 80% of the calls are randomly sampled over 100 iterations. Sub-figure (a) shows the percentage of calls that were correctly classified. Sub-figure (b) shows the deviation of the number of fitted clusters from the actual number of individuals (females). The box plots describe the median, 1st and 3rd quantiles and the range of the data for each site and algorithm. Credit: [15, Fig 4], Re-used with permission from "John Wiley and Sons"

Limitations of Unsupervised Acoustic Classification

One major drawback of K-medoids is the fact that **the number of clusters must be pre-determined** [39, p.248 "Limits of K-means"] and must be provided as an input parameter. To overcome this limitation, the silhouette score can be calculated for various numbers of clusters and the value that maximises it is chosen. This approach is employed by the authors in [15]. However, this can be computationally expensive for large datasets, as the algorithm must converge multiple times.

For Gaussian mixture models, the number k of Gaussian distributions must be known beforehand [39, p.260 "Gaussian Mixtures"], posing a similar problem.

Affinity propagation does not need the number of clusters specified. However, this

does not mean it is the panacea. While in the experiments, it outperformed the other algorithms in the percent correct metric and the deviation of number of clusters from the number of actual individuals, it actually had a **worse mean NMI score** for N=30 and N=40 individuals, as shown in Figure 2.7. Moreover, from sub-figure (a) of Figure 2.8, we can see that for the Mailiau Basin site, the percentage of correct classifications varied a lot more for Affinity (shown in yellow) than Gaussian and K-medoids, evident from the larger interquartile range.

In addition, notice in Figure 2.7 that, for all three algorithms, the mean percentage of correctly classified females decreases as the number of females increase, with Affinity dropping from 75% for N=10 to 65% for N=50.

Hence, we can deduce that there is no "one size fits all" clustering algorithm for automatic acoustic classification and the **clustering performance depends on the task at hand and the input data provided.**

Furthermore, the authors admit that a caveat of their experiments is that the **calls were of high quality** (SNR[3] > 10 dB) compared to what would be captured using recorders deployed at fixed locations.

Finally, they state that the acceptable level of error in unsupervised classification for PAM applications depends on the research question. In situations where assumptions about individual identity permit minimal error, these methods may not be suitable.

2.2.3 The "ecostats-rshiny" App

In this section, we review the original version of the app "ecostats-rshiny", developed by "the-tarin" and available at github.com/the-tarin/ecostats-rshiny. Written in Shiny, a web framework for the R language discussed in depth in section 2.4.1, the app has since been rewritten and improved as part of this dissertation. However, we will focus on its initial functionality here.

The app has been purpose-built to tackle the challenge of identifying calls from the same individual within passive acoustic monitoring data. In particular, it incorporates a

[3]https://en.wikipedia.org/wiki/Signal-to-noise_ratio

visualisation which facilitates matching calls by comparing their estimated bearings, i.e. the angles from which they originate.

Input Data

mic_id	lat	lng
1	14.219238	106.603588
2	14.219238	106.622143
3	14.219238	106.640698
4	14.219238	106.659253

Table 2.3: Example of microphone location data that ecostats-rshiny accepts as input. "lat" corresponds to latitude, "lng" corresponds to longitude. Each microphone is uniquely identified by "mic_id".

recording_ID	mic_ID	measured_call_datetime	measured_bearing
1	1	2023-01-01 04:31:19	2.20535531915042
2	3	2023-01-01 04:16:38	5.74755987091546
3	1	2023-01-01 04:43:41	5.13921089450594
4	4	2023-01-01 04:10:23	0.732737313432567
5	1	2023-01-01 04:22:13	1.13986648899937
6	3	2023-01-01 04:58:28	5.40912155694918
7	4	2023-01-01 04:31:48	5.70646496100251
8	2	2023-01-01 04:48:42	2.41883397510063
9	2	2023-01-01 04:34:56	0.423823994441011
10	2	2023-01-01 04:31:19	4.27983374423573

Table 2.4: Example of the recordings csv format which "ecostats-rshiny" expects. For each recording, the mic_ID, the time of arrival of the sound (measured_call_datetime) and the estimated angle it was heard from (measured_bearing) must be specified.

Ecostats-rshiny takes as input two CSV files, the first one containing the microphone geolocation data, and the latter containing metadata about the calls captured by the microphones.

An example of microphone location data is shown on Table 2.3. The latitude and longitude must be specified for each microphone used in the acoustic survey.

Table 2.4 illustrates the format of recording CSV data that ecostats-rshiny accepts. Specifically, for each call uniquely identified by "recording_ID", three parameters must be supplied :

1. The microphone this call was recorded from, specified by the "mic_ID" column
2. The time the microphone captured this call, given by the "measured_call_datetime" column
3. The estimated bearing from which the microphone captured the call, defined in the "measured_bearing" column

Aside: How are these parameters extracted?

As discussed briefly in "SECR and Acoustic Surveys", section 2.1.3.1, the raw recording data is processed using specialised software that enables researchers to detect and segment calls by visualising spectrograms and listening to the audio. A popular choice for this software is PAMGUARD [41]. When each recording unit contains multiple microphones, PAMGUARD can also compute the estimated bearing of a given call. The researcher can output the recording metadata into a CSV file and input it to "ecostats-rshiny".

The "ecostats-rshiny" Workflow

The user selects and uploads the microphone and recording csv files, via the sidebar shown on Figure 2.9. Afterwards, the user can filter the calls via the datetime sliders in the sidebar, such that they can focus on processing a few calls at a time.

Figure 2.10(a) displays what the user is presented with after the input data has been loaded.

On the left side, the locations of the microphones are displayed as red dots on a map, using the coordinates provided in the input data. Hovering over a red dot, reveals a popup box denoting the mic_ID of that microphone.

On the right side, the recorded calls appear on a table which is interactive. They can be sorted by any column of the table by clicking on the column headers. Moreover, multiple calls can be selected from the table by clicking on them. Then, arrows are drawn on the map based on the estimated bearings of the selected calls, as shown on Figure 2.10(b).

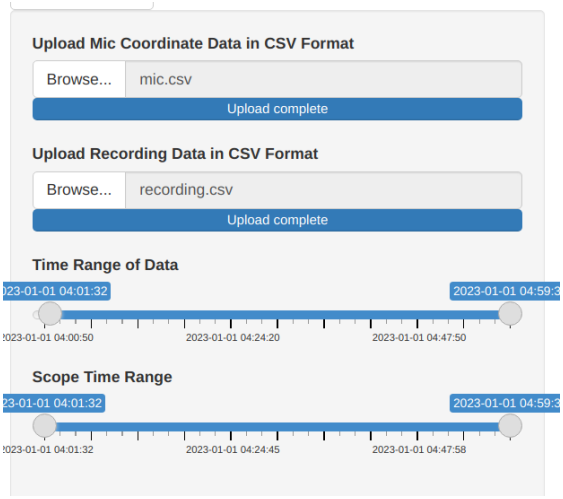
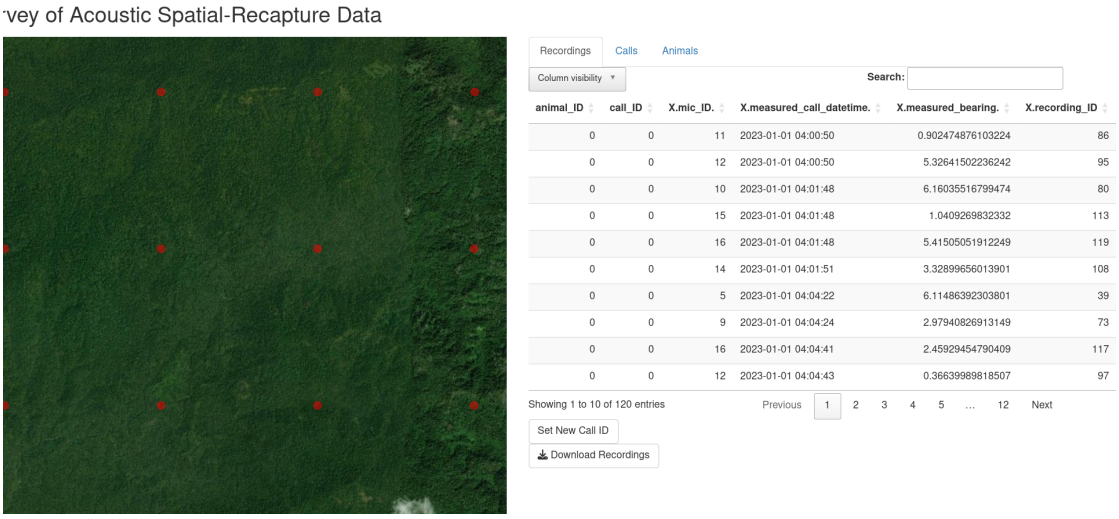


Figure 2.9: Sidebar of ecostats-rshiny where the user uploads the input data i.e. the microphone location coordinates and the recording data. Two datetime sliders are provided to filter the recording data based on measured_call_datetime. The top slider applies an initial filter to all calls, while the second slider refines this selection, allowing users to focus on a specific time range within the initially filtered calls.

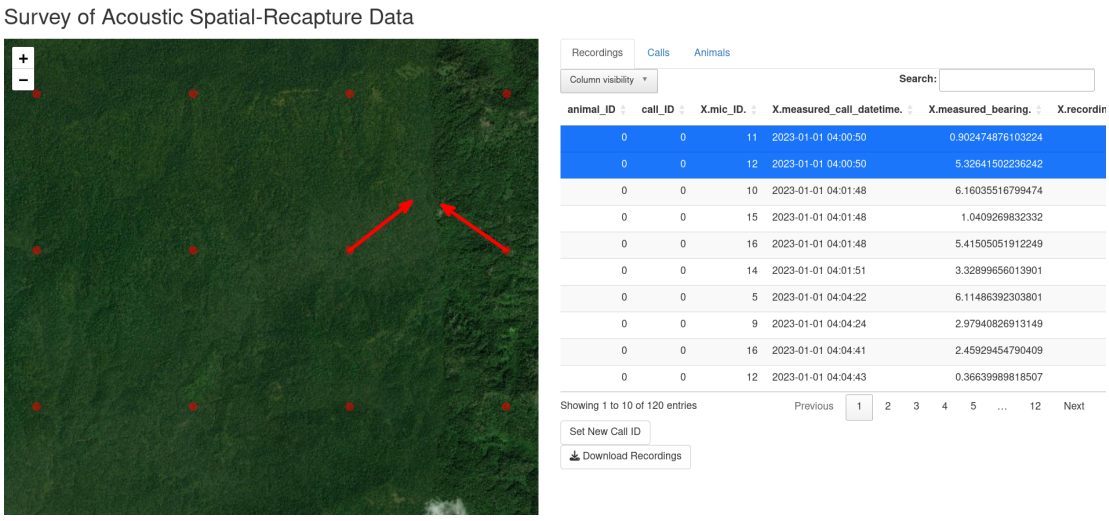
How Are Calls Matched?

By **observing** whether the arrows **point** to the **same direction** and **determining** the **time gap** between calls, the user can find **which calls** potentially belong to the **same** vocalising **individual**.

Notice how the recordings table in Figure 2.10 contains two extra columns: animal_ID, and call_ID. These are automatically added by the app and they are used for call matching. Specifically, when the user has selected two or more calls, as in Figure 2.10(b), they can click the "Set New Call ID" button, on the bottom right, which automatically assigns the same call id on the selected rows.



((a))



((b))

Figure 2.10: Default View of "ecostats-rshiny". The main interface features a map on the left, displaying the geographical positions of recorders as red dots. On the right, a tabbed layout initially shows all recorded calls in a table. Sub-figure (b): Users can select multiple rows in the table to visualise their estimated bearings. Clicking "Set New Call ID" updates the call_ID values of the selected calls to be the same.

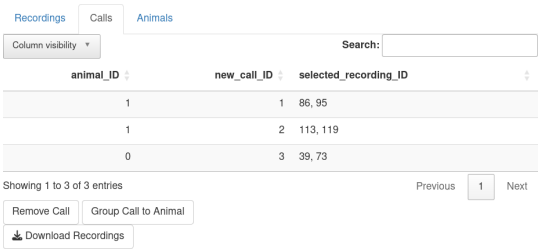


Figure 2.11: Calls Tab View of "ecostats-rshiny". In this screen, the matched recordings and their corresponding call (group) id are displayed in a table format. The user can select multiple rows, visualise their bearings, and click "Group Call to Animal" to assign the same animal ID to them. Clicking "Remove Call", removes all the selected call groups.

Clicking on the "Calls" tab, on the right side, reveals the UI shown in Figure 2.11. This displays a table of matched recordings, under "selected_recording_ID", and their associated call id, under "new_call_ID". By selecting one or more rows, the bearings of the selected call groups are plotted on the left side, as previously demonstrated. The user can then click "Group Call to Animal" to assign the same animal_ID to the selected rows. This is necessary because subsequent calls may belong to the same vocalising individual, and there is no way to mark these calls with the same call_ID as previous calls in the recordings table.

Finally, clicking on "Download recordings", updates the table shown in Figure 2.10 with the modified animal_ID and call_ID values and saves it as a CSV file. This file can be processed further and be used for ecological statistical analysis, such as population density estimation using SECR models.

Limitations of "ecostats-rshiny"

Below, we have identified some of the limitations which prompted us to rewrite the app and improve it.

1. Unintuitive Call matching

When the user matches calls under the Recordings tab, as shown in Figure 2.10(b), there is **no separation between already matched calls and unmatched calls**. One way to distinguish between them is to check if call_ID is zero. However, we argue that that is easy to miss, when there are multiple calls across many pages, as in Figure 2.10(b).

Alternatively, the user can manually filter the matched calls via the sliders, by checking the datetime of the last matched call. Again, this is quite an involved process.

Moreover, when the user selects calls on a page other than the first and clicks "Set New Call ID", the **table updates and resets back to the first page**. The user has to remember the last page they were on to continue matching calls.

Furthermore, suppose that the user forgets to select a call before setting the call ID. The only choice the app offers is to select a row from the Calls tab page (Figure 2.11) and to click "Remove Call". This sets the call_ID of all recordings associated with that row back to zero, effectively **removing the whole group**, i.e. there is no option to remove just a single call from a group.

To make matters worse, when 'Group Call To Animal' is clicked, the **app crashes** sometimes, requiring the user to match all the calls again.

2. Cluttered UI

The entire UI of "ecostats-rshiny" is presented on a single web page. The map is displayed on the left, while the tabbed layout containing the recordings and call groups is shown on the right, with the sidebar positioned below them. To filter the data, the user must scroll down, apply the filter, and then scroll back up, which hides the filter from view. This can lead to the user **forgetting** that a **filter** has been **applied**. Additionally, if the table contains many columns, it can overflow outside the viewport of the browser window, meaning that **important data attributes** might be **hidden from view.**

3. Maintainability

The entire app consists of a **510-line file** with **few comments**, and many interconnected components. This makes **debugging challenging**, especially when stack-trace error messages do not clearly indicate which line caused the failure. Additionally, this complexity makes it difficult to add new functionality and **hinders future maintenance** by other developers.

4. Reliance on Bearings

The "ecostats-rshiny" app is designed to match calls from the same individual based

on estimated bearings. However, these bearings can be inaccurate. The data shown in Figure 2.10(b) is simulated. When we tested the app with real-world data, the estimated bearings proved to be less effective for matching calls, so we had to rely on comparing spectrograms instead.

2.3 Literature Gap

To our knowledge, there exists no human-in-the-loop, visualisation-based tool for passive acoustic monitoring surveys to match and localise animal calls heard on multiple microphones, which is fully-functional and user-friendly.

As discussed in section 2.2:

- machine-learning based methods used to classify and match calls, are not generalisable, and they do not observe the same accuracy as human experts manually shifting through recordings, visualising them and labelling them.
- Koe primarily targets studies whose purpose is syllable classification, e.g. birdsong analysis. It provides no functionality to match calls based on bearings, and it doesn't take into account the location of the recorded calls.
- "ecostats-rshiny," although useful for matching calls and identifying individuals based on estimated bearings, is unfinished, buggy, and not user-friendly.

2.4 Background Information

2.4.1 Shiny Framework and Reactive Programming

In this section, we present the Shiny framework and discuss why its reactive programming approach to building web apps is both useful and powerful.

Both "ecostats-rshiny" and Vocomatcher have been developed using the Shiny framework.

Shiny is a web application framework written in R[4]. It allows developers to build full-

[4]Click here to learn more about the R programming language.

stack interactive and dynamic applications that run in a web browser using only R i.e. no knowledge of HTML, CSS, JavaScript is necessary. Shiny comes in handy for many data scientists/statisticians who are already familiar with the R language, and they want to quickly build an interactive data mining tool, a simulation or any other dynamic visualization[5].

Moreover, Shiny hides away the complexity of frontend development by offering many useful GUI components out of the box, such as tables, plots, date pickers, dropdowns, which are highly customisable and extendable.

Furthermore, the server logic of a Shiny app is expressed via reactive programming. What is reactive programming? To answer that question, let's build a simple app that asks the user for their name and greets them.

Here is the skeleton of every Shiny app:

```
#Import shiny framework library
library(shiny)

#Define all frontend interface in ui
ui <- fluidPage(
...
)
# Define all backend logic in server
server <- function(input, output) {
...
}
# Run the application
shinyApp(ui = ui, server = server)
```

Shiny follows the client-server model. All code within the *ui* function runs on the client's web browser, while the backend server logic resides in the *server* function. The former defines the look of the app. It consists of layouts, which define the high-level visual structure and UI widgets/components. Building UI in Shiny is nothing more than creating a hierarchy of layouts and components. In the code snippet above, a fluid page layout is used for the UI which consists of rows which in turn include columns. Other layouts are also available, but for our example, `fluidPage` will suffice.

[5]Navigate to https://shiny.posit.co/r/gallery/#feature-demos to see some examples.

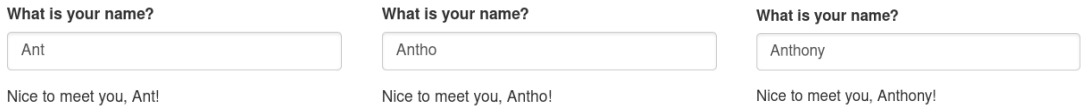


Figure 2.12: Shiny reactivity example: The user enters their name, and as they type it in, the greeting text automatically updates.

The server variable is a function where we explain how to *react* to changes in the UI and manipulate/update it. The `input` parameter is a key-value pair data structure where the key is a unique ID corresponding to any input UI component e.g. text box, slider, radio button and the value is what the user typed-in or selected for that component. Hence, it is used for reading input. Conversely, the `output` parameter is used for updating all the output UI components e.g. a plot, a table, some text. These two variables are passed as arguments to the `shinyApp` function, which builds the app and launches it on a local HTTP port, accessible via a web-browser.

For the UI, we use a text input field, with the id `nameIn`, where the user enters their name and a text output field, with id `greetingOut`, where the greeting shows up. Here is the code:

```
ui <- fluidPage(
  # Text input field for the user's name
  textInput("nameIn", "What is your name?"),
  #Text output which greets the user.
  textOutput("greetingOut")
)
```

Finally, in the back-end logic, we make `greetingOut` re-render its text whenever the input name changes using the `renderText` function as shown below:

```
# Define business logic in server
server <- function(input, output) {
  output$greetingOut <- renderText({
    paste0("Nice to meet you, ", input$nameIn,"!")
  })
}
```

The simple app we built is shown in Figure 2.12. Notice how while typing the name

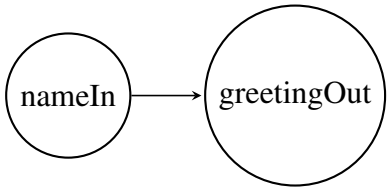


Figure 2.13: Reactive dependency graph of the toy-example Shiny app presented in section 2.4.1. The greeting text will automatically re-render when the input name changes.

"Anthony", the greeting text automatically updates. **We don't need to tell the output when to update or how to update, since Shiny automatically figures that out!** This is what reactive programming is all about.

The main premise of reactive programming is to construct a graph of dependencies such that when an input changes, all related outputs are automatically updated. To visualise the order of execution of any Shiny app, we can use directed node-link diagrams. Such a diagram is shown in Figure 2.13 for our example app. This diagram denotes that `greetingOut` will need to be re-rendered whenever `nameIn` is changed. In other words, `greetingOut` has a reactive dependency on `nameIn`.

Why is reactive programming so powerful, though? To answer that, let's consider the alternative: imperative, event-driven programming. In such a paradigm, it is up to the programmer to explicitly manage the state of the application. For example, instead of telling Shiny that the greeting should be re-rendered whenever the input name changes, we would have to explicitly encode *how* and *when* to update it. In JavaScript, this would be achieved by hooking an input event listener to the text box that would call a custom function of ours (a callback) which would update the greeting to reflect the new name. This implies that there are more things to keep track of during development, which can consequently lead to bugs, especially if multiple events can change the state.

To conclude, Shiny allows one to quickly develop a complicated web-app without necessarily worrying about the underlying web technologies involved. Its clean and declarative approach of handling events via reactive programming simplifies the business logic and enables developers to create dynamic, responsive user interfaces that automatically update in response to user input or data changes. For these reasons, and others discussed later on, we opted to implement Vocomatcher in Shiny.

CHAPTER THREE

METHODOLOGY

This chapter outlines the software engineering methodology we followed to develop Vocomatcher, and explains why it was chosen over a more agile approach.

3.1 Waterfall Methodology

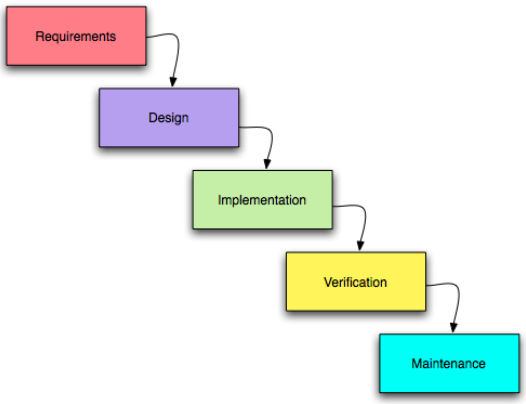


Figure 3.1: The waterfall model. Picture credit: https://www.umsl.edu/ hugheyd/is6840/waterfall.html

To develop Vocomatcher, we followed the Waterfall methodology [42]. This comprises five main phases, as shown in Figure 3.1 : Requirements, Design, Implementation, Verification, and Maintenance.

Each subsequent chapter focuses on one of these phases and provides a detailed description of the methods employed in that phase.

In the requirements phase, the needs and wants of the client are noted down. Detailed documentation is created to capture the client's expectations and constraints, ensuring that all requirements are clear and unambiguous.

In the design stage, various solutions are brainstormed to address the client's needs and expectations. This includes creating user interface sketches, proposing algorithms to tackle specific problems, and selecting suitable software tools for the project. These design ideas are then presented to the client for feedback. Based on the client's input, adjustments are made, and if need be, the project's requirements are updated.

In the implementation phase, the final candidate design and the requirements specification are used to code the software solution.

In the verification phase, it is verified whether the implementation fulfils the project's requirements. For example, this can be done through unit testing and integration testing or beta testing with the client. If some requirements are not met, the developer tries to identify where the issue lies.

Once the software satisfies all requirements and passes validation, it enters the maintenance phase. During this stage, small updates may be released to address minor bugs and to refactor/optimise the code.

3.2 Why Was the Waterfall Methodology Chosen?

We chose to follow the Waterfall methodology because our client had a clear vision of the improvements they wanted over "ecostats-rshiny" and due to time constraints. Our client takes annual leave during the summer, which limits the opportunity for iterative feedback throughout the development process. Therefore, we opted for the Waterfall approach over a more Agile method to ensure that the software solution's requirements were clearly understood from the outset, thereby reducing the need for major modifications later on.

CHAPTER FOUR

REQUIREMENTS SPECIFICATION

In this chapter, we discuss the methods used to capture the requirements necessary to build Vocomatcher and why these specific methods were chosen. Moreover, we present the final version of the elicited requirements.

4.1 Approach to Requirement Collection

4.1.1 Contextual Inquiry

The process of capturing requirements for Vocomatcher involved combining direct client interaction, independent analysis, and continuous refinement. We initiated our requirements gathering through a contextual inquiry [43, Chapter 3] session with the client.

We chose to conduct a contextual inquiry because the client, being an expert in their field, could provide valuable insights into the precise requirements needed. Additionally, as the client had been using "ecostats-rshiny" for some time, they already had specific improvements in mind.

During this contextual inquiry, the client provided an overview of acoustic surveys, explaining the detection, segmentation, and classification of Gibbon calls. Moreover,

they walked us through their current software pipeline for generating input data for "ecostats-rshiny" and demonstrated the functionality of the existing application.

As part of this inquiry, they highlighted the limitations they faced while using "ecostats-rshiny". These challenges are detailed in section 2.2.3. The client also shared their initial ideas for potential improvements. We meticulously documented their likes, dislikes, and suggestions regarding the current application.

4.1.2 Independent Evaluation of "ecostats-rshiny"

Following the client demonstration, we performed an independent evaluation of "ecostats-rshiny", evaluating its usability, performance, and maintainability. This allowed us to identify additional issues that the client may not have mentioned, and to assess the feasibility of modifying the existing application versus developing a new one from scratch. After careful consideration, we concluded that building a new application would be more efficient than tweaking the existing one. We communicated this finding to our client, who agreed with us and expressed willingness to provide input for the new application.

4.1.3 Requirements Classification and Prioritisation

Throughout the requirements gathering and design stages, we maintained regular communication with the client via email. This allowed us to seek clarification on ambiguous points and continuously refine our understanding of the requirements. We opted to collect as much information as possible during the initial stages of the project, as our client informed us of their limited availability during the summer period due to planned annual leave.

The requirements underwent several iterations, particularly during the design phase. As part of this process, we categorized the requirements into two main types: functional and non-functional. Functional requirements describe specific behaviours or functions that the system should exhibit, while non-functional requirements specify criteria that can be used to judge the operation of a system, rather than specific behaviours.

We then ranked these requirements based on three key factors:

- The importance of the requirement to the client
- The novelty of implementation, which we assessed by researching the availability of existing libraries or tools that could aid in development
- The time remaining until the dissertation submission deadline

This approach allowed us to focus our efforts on the most critical and feasible features within our time constraints.

The final version of the requirements, which forms the foundation for Vocomatcher's development, is presented below.

4.2 Requirements Gathered

Please note that in the "Priority" column, a lower number indicates a higher priority.

ID	Name	Type	Priority	Description
1	Recordings CSV File Upload	Functional	1	The user can select and upload the recording data.
2	Microphone CSV File Upload	Functional	1	The user can select and upload the microphone location data.
3	View uploaded data	Functional	3	The user can view the uploaded CSV files in table format to double-check they are the correct files.

4	Select date	Functional	2	The user picks from a list which date to work on and only the calls from that specific date are shown on the app.
5	Filter data by date	Functional	1	All calls captured on dates other than the user-selected one must be filtered out.
6	Separation of matched and unmatched calls	Non-functional	1	Calls which have already been matched must be visually separated from unmatched calls.
7	Select multiple calls	Functional	1	The user can select multiple calls to check their bearings and/or match them.
8	Intuitive way to check time difference of calls	Non-functional	1	There must be an easy and intuitive way to check the time difference between calls.
9	Visualising microphone coordinates	Functional	1	The microphone coordinates must be visualised as marks on the bearings plot.
10	Bearing map auto-update	Functional	1	The bearings plot visualization must automatically update every time a new call is selected or deselected.

11	Visualising call bearings	Functional	1	The bearings of calls must be visualised as arrows on the map pointing to the estimated vocalisation source.
12	Microphone ID popup on bearings plot	Functional	2	Hovering over a microphone mark on the bearings map should show a pop-up indicating the ID of that mic.
13	Recording ID popup on bearings plot	Functional	2	Hovering over an arrow of a selected call on the bearing map should show a pop-up of the recording ID.
14	Colour arrow based on sex on bearings plot	Functional	1	The arrow colour must be different for male and female individuals.
15	Create new call group	Functional	1	The user can create a new call group based on the selected calls by clicking a button.
16	Add calls to an existing call group	Functional	1	Selected call(s) can be inserted to an existing call group by clicking a button.
17	Modify call group	Functional	1	Calls can be removed from a call group.

18	Separation of call groups	Non-functional	1	There should be a way to visually distinguish between call groups such that calls can be added/removed effectively.
19	Filter unmatched calls by time of arrival	Functional	2	The user can filter the unmatched calls by time of-arrival via a slider.
20	View spectrogram of unmatched call	Functional	1	Each row in the unmatched calls table must contain a column where the spectrogram of said call is displayed.
21	Compare multiple spectrograms	Non-functional	2	The user can compare the spectrograms of selected calls.
22	Zoom in to spectrograms	Functional	3	The user can enlarge spectrograms to compare them better.
23	Matched calls (Call groups) Export to CSV/TXT	Functional	1	The matched calls (call groups) can be exported to a CSV file or TXT file.
24	ascr R package export format	Non-functional	1	The exported CSV/TXT file must abide to the capture history input format specification of the ascr R package.

25	Responsive UI	Functional	1	The app's UI widgets must resize automatically to fit the viewport of the browser window.
26	Eyes beat memory	Non-functional	1	Wherever possible, the UI must be designed in such a way that the user's perception is used to match calls rather than relying on their memory.
27	Different screens for different functionalities	Non-functional	1	The different functionalities of the app, i.e. uploading, filtering, matching, exporting calls must be separated to different screens such that the user is not overwhelmed by everything being on one screen.
28	Testing	Non-functional	2	The app must be well-tested with different inputs and input lengths. Edge cases must be considered.
29	Useful error messages	Functional	2	The app should display useful error messages to the user.

30	Compare call group and unmatched calls	Functional	3	The user should still be able to compare the bearings and the spectrograms between matched and unmatched calls even if they are separated.

CHAPTER FIVE
DESIGN

This chapter presents the UI mock-ups of Vocomatcher, discussing the rationale behind key design decisions, the requirements they fulfil, and the limitations of "ecostats-rshiny" they address. We focus particularly on the new design for call matching, providing an in-depth explanation of its functionality and highlighting its advantages over the "ecostats-rshiny" user interface.

5.1 Wizard UI Pattern

To address the "Cluttered UI" limitation of "ecostats-rshiny", and satisfy software requirement #27, we divided the UI into separate screens, with each screen dedicated to performing a single function. The screens in order are : Import CSVs screen, Select Date screen, Match Calls screen, and Export Matched Calls screen.

To link the screens together, we apply the Wizard UI pattern [44, p. 42]. This allows users to navigate between screens using "Previous" and "Next" buttons, provided all necessary actions on the current screen are completed. This minimises the room for error, as the user cannot "jump ahead" and interact with components which are dependent on previous actions being completed.

Furthermore, said pattern forces the user to process the input data in a prescribed manner, which simplifies the overall task, since each subtask can be dealt with in a discrete mental space.

5.2 UI Mock-ups

This section presents the finalized UI mock-ups, created after extensive collaboration with the client.

5.2.1 "Import CSV" Screen

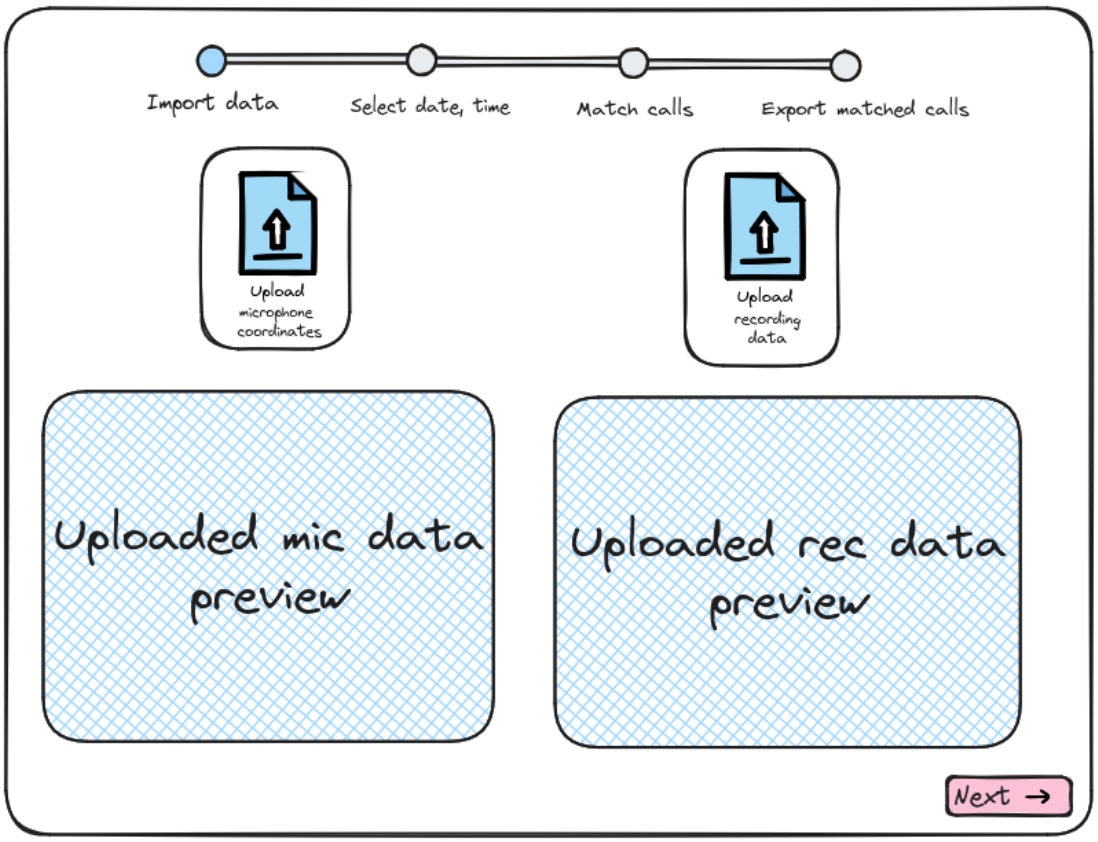


Figure 5.1: "Import CSV" screen of Vocomatcher. The user selects and uploads the microphone coordinates and the recorded calls CSV files. The user can view the uploaded files to make sure they are the correct ones.

This is the first screen of the app. The user uploads two CSV files. The first one contains the location coordinates of the microphones used in the study. The latter, contains metadata regarding the captured calls.

Two previews are shown on screen such that the user can make sure they uploaded the correct files.

After the required files are uploaded, the "Next" button becomes available, which when clicked shows the "Date Select" screen.

5.2.2 "Date Select" Screen

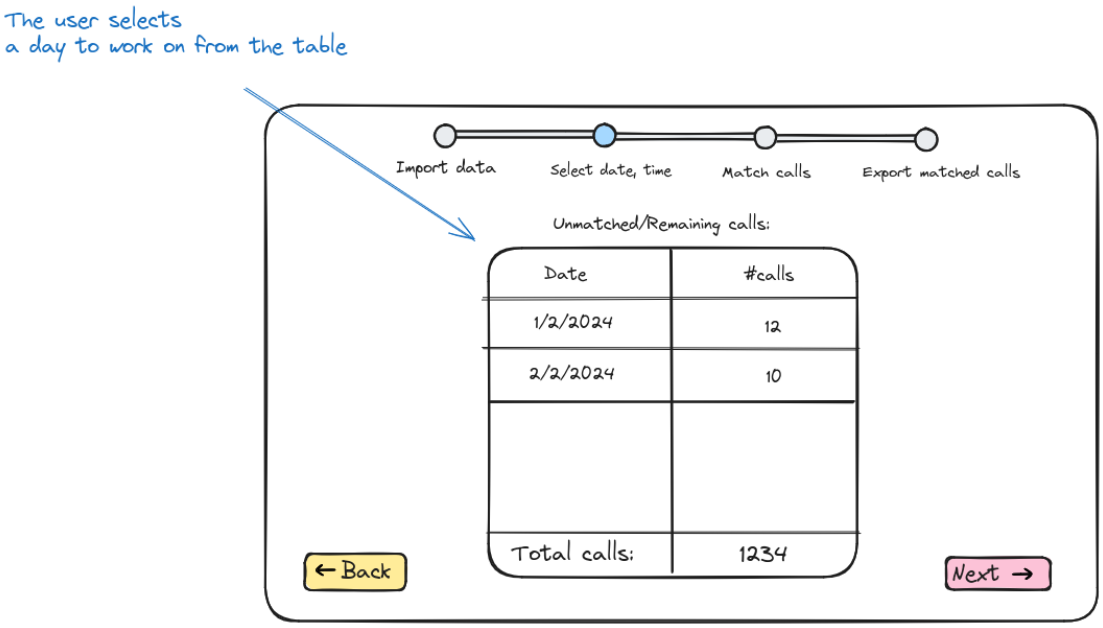


Figure 5.2: "Date Select" screen of Vocomatcher. The user picks a date from the table to only process calls from that date.

On this screen, calls are grouped by date and displayed in a table. The user selects a row to process only the calls from that specific date.

This approach ensures that the user matches a manageable number of calls at a time. We believe this method is superior to "ecostats-rshiny," which loads all available calls and requires the user to filter them with a datetime slider. If the user accidentally closes Vocomatcher or the app crashes, fewer matched calls will be lost.

After the user selects a date, the "Next" button becomes available, which when clicked reveals the "Match Calls" screen.

5.2.3 "Match Calls" Screen

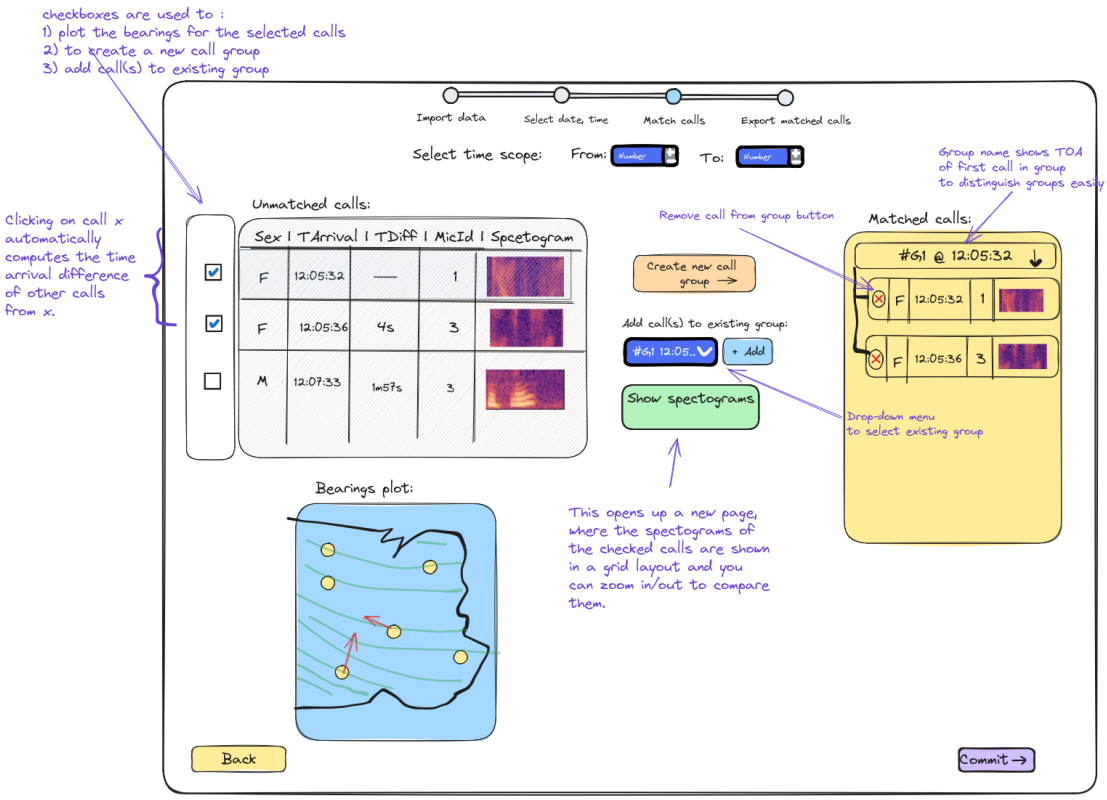


Figure 5.3: "Match Calls" screen of Vocomatcher. Users compare calls via spectrograms, and the bearings map to form call groups. Calls are selected via checkboxes, and time differences are calculated by clicking on unmatched calls. Call groups are managed on the right, allowing users to create, add, or remove calls. "Show Spectrograms" opens a detailed comparison window.

This screen shows all the unmatched calls on a table on the left and all the matched calls on a collapsible list on the right. Above those, there exists two spinners which filter the unmatched calls by the selected time scope. Below the table is the bearings map, the same one found in "ecostats-rshiny", which plots as vectors the estimated bearings of selected calls.

The unmatched calls table comprises four columns: the sex corresponding to each calling individual, the time each call was heard (TArrival), the time difference of a selected call and others (TDiff), the microphone which heard the call (MicId), and the call itself visualised in a spectrogram image.

5.2.3.1 How Are Calls Matched?

1. The user clicks on a row from the unmatched calls table to select it. This populates the TDiff column with the time difference between the selected call and all other calls in the table.

2. The user then looks for calls which have a small TDiff value and the same sex as the selected row.

3. Using the checkboxes to the left of the unmatched calls table, the user picks the calls found in the previous step.

4. The bearings of the checked calls are automatically plotted on the bearings map.

5. The user checks whether the vectors point to the same direction in the bearings map.

6. Should the user need more evidence that checked calls belong to the same individual, they can click the "Show spectrograms" button. This opens up a new page, where the spectrograms of the selected calls are shown in a grid layout and the user can enlarge them to compare them more thoroughly.

7. If the user is happy with their matching, they can click "Create new call group", which creates a group of the checked calls in the matched calls list.

8. If instead the checked calls pertain to a previously created group, they can select an existing call group and click "Add" (shown in blue in the middle of Figure 5.3).

9. If a call was added in the wrong group, the user can click the "X" button next to the call in the "Matched calls" list to remove it back to the "Unmatched calls" table.

Figure 5.4 illustrates clearly how call matching works in the form of a flowchart.

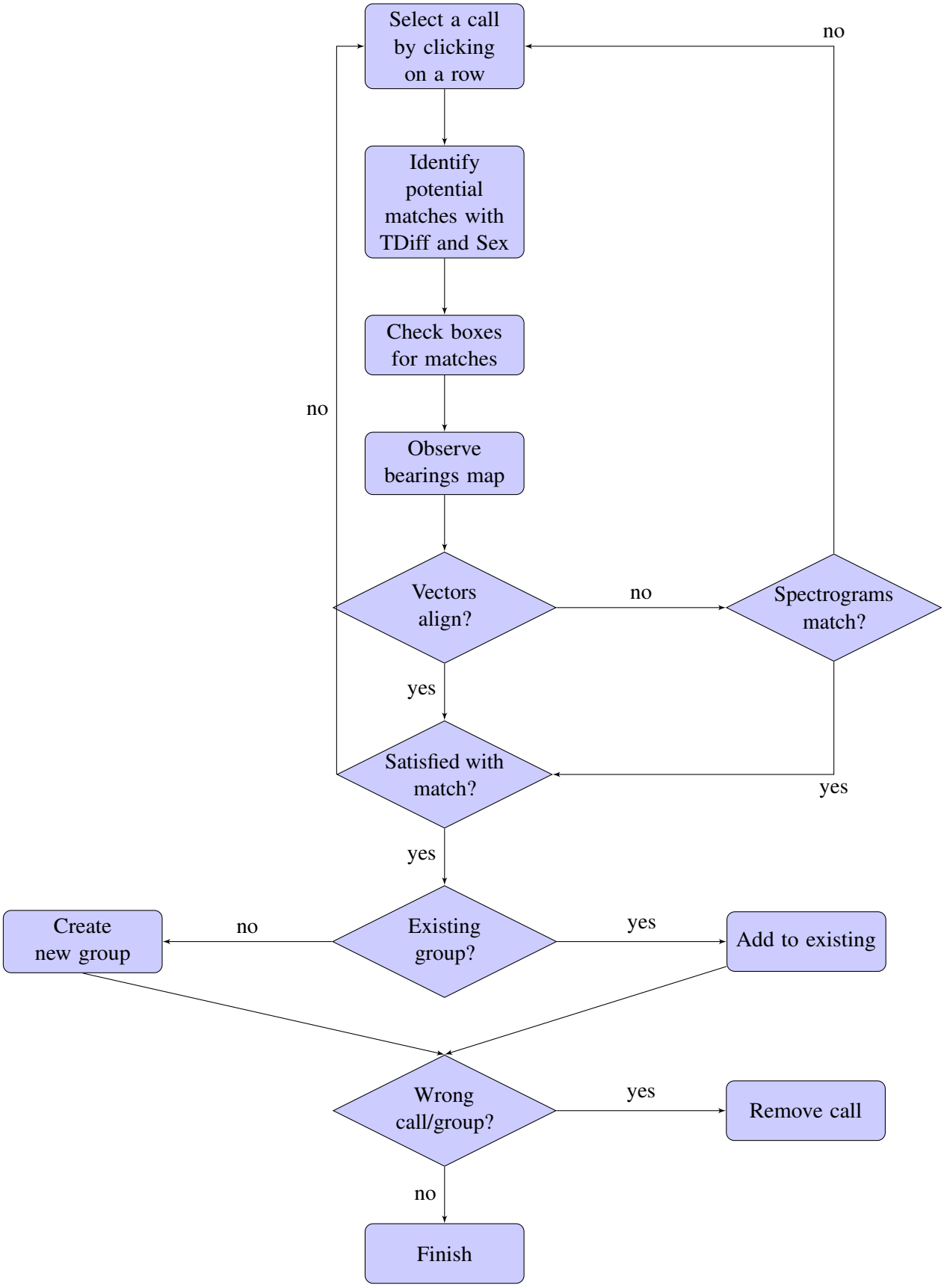


Figure 5.4: A flowchart describing how to match calls in Vocomatcher.

5.2.3.2 Why is this design better that the one of "ecostat-rshiny"?

This design incorporates software requirement #6 i.e. it clearly separates unmatched from matched calls, and it provides an intuitive way to add and remove calls from existing groups. The user need not manually set a Call ID; this is handled exclusively by the app.

Furthermore, this design addresses limitation #4 of "ecostats-rshiny" while fulfilling requirements #20, #21, #22. The user is offered an additional tool to compare calls inspired by Koe's unit table view (Figure 2.6), spectrogram images of the calls!

Moreover, the user need not manually compute the time difference between calls, this is done automatically by just clicking on a row in the unmatched calls table, satisfying requirement #8.

5.2.4 "Export Calls" Screen

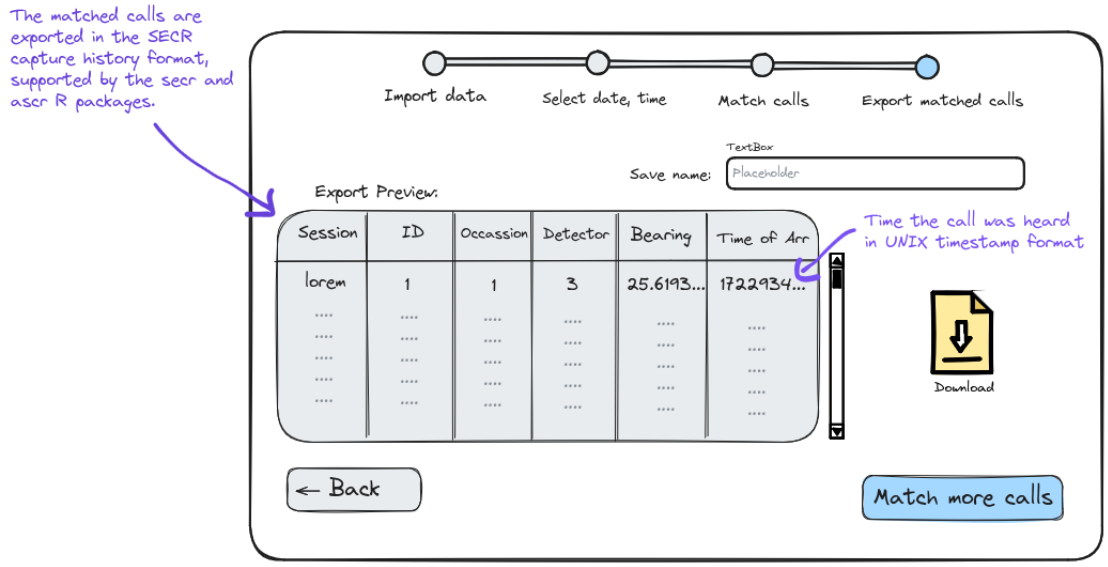


Figure 5.5: "Export Calls" screen of Vocomatcher. A capture history table is created based on the matched calls. The user can type a name for the table and export it to a TXT file, which is compatible with the `secr` and `ascr` R packages.

Here, all the call groups that were created in the previous screen are shown in a table which follows the capture history format of SECR models, discussed in section 2.1.3.1.

To be more precise, the format shown on screen is the one accepted by the `secr` and `ascr` R packages. The former is used for fitting spatial capture recapture models on any data, while the latter is tailored to acoustic data. The additional columns, *Bearing* and *Time Of Arrival*, improve the density estimate of the model.

On this screen, the user can enter a name for the data to be exported and click "Download" to save it. The data is exported as a text file that is directly readable by the `read.capthist` function of the `secr` package, which produces a capture object. This object can then be made compatible with `ascr` using `convert.capt.to.ascr`. For more details regarding the data input format supported by these packages, see [45, "Capture data format"] and [46, "Compatibility with `secr`"].

5.2.4.1 Comparison with "ecostats-rshiny"

"ecostats-rshiny" exports the matched calls in a CSV file which is just the uploaded recordings CSV file with two extra columns: Animal_ID, and Call_ID. It is then up to the researcher to process this file and transform it into a format used for SECR analysis.

Our client informed us that they only intend to use Vocomatcher for SECR analysis. Therefore, it makes sense to export the matched calls in a format directly compatible with the R packages they use.

CHAPTER SIX

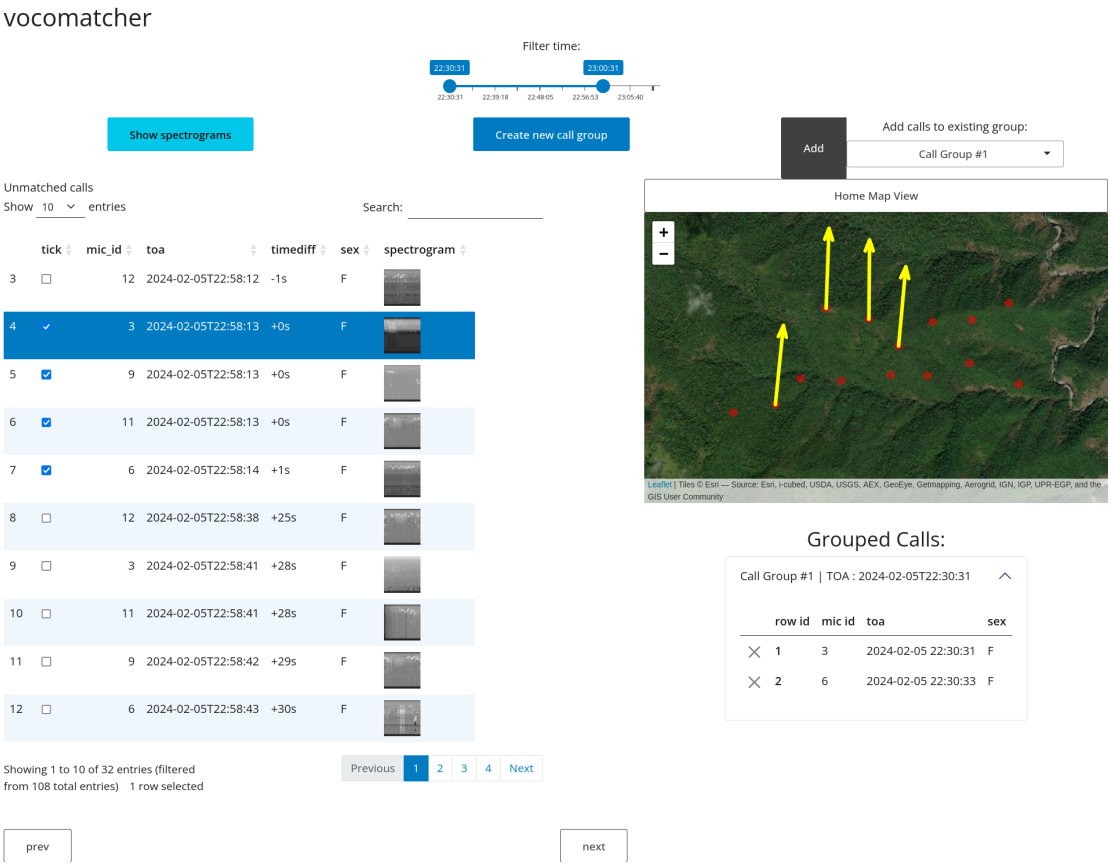
VOCOMATCHER - SOFTWARE IMPLEMENTATION

In this chapter, we present the final look of Vocomatcher, discuss the software tools and techniques used in its development, and provide implementation details, including the challenges faced and how they were resolved.

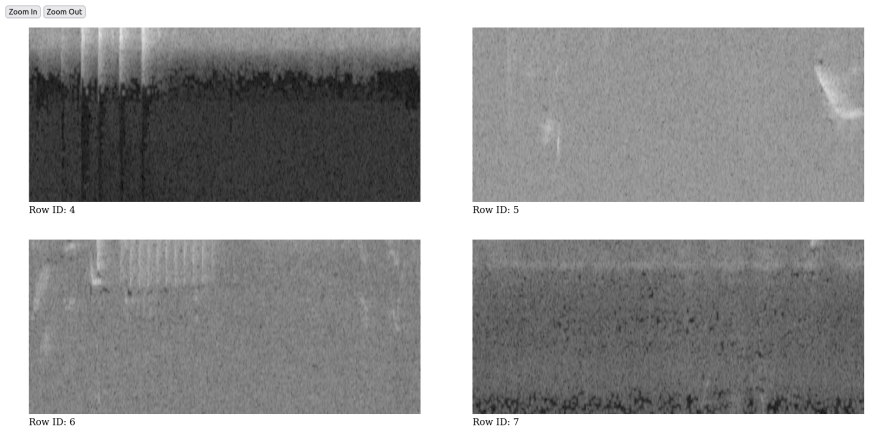
6.1 Final Look

Screenshots of the "Match Calls" and Spectrogram grid view pages are shown on Figure 6.1. To match calls, the exact same steps described in section 5.2.3.1 apply.

The final design of the calls page closely resembles its mock-up. There are few minor differences. First, we reverted to using a slider for filtering the time scope, as this was easier to implement. Second, the bearings plot is positioned to the right of the unmatched calls table and above the grouped calls. This was done because if it was positioned below them, the user would have to scroll up and down to check the bearings. Third, the arrows drawn on the map are colour coded according to the sex of the call they correspond to (yellow for female, cyan for male) in case the user does not check the sex value on the unmatched calls table.



((a)) "Match Calls" screen of Vocomatcher



((b)) Spectrogram grid view of Vocomatcher

Figure 6.1: Demonstration of call matching using Vocomatcher. Sub-figure (a): The user clicks on row number 4 which populates the time difference column. They check all rows with a small TDiff and the same sex and observe the direction of the arrows on the bearings map. The arrows are color-coded according the sex (yellow for female, cyan for male). Then, the user clicks on "Show Spectrograms" (top left). Sub-figure (b): The spectrograms of the checked calls are shown in a grid layout. The user can zoom in/out via the buttons at the top left to inspect the spectrograms more closely.

6.2 Implementation Details

Vocomatcher is implemented in the Shiny web framework, presented in section 2.4.1, using the Golem toolkit.

We chose to code Vocomatcher in the same framework as "ecostats-rshiny" for several reasons:

- To take advantage of the reactive programming paradigm of Shiny, which significantly speeds up development time
- To easily port code from "ecostats-rshiny" with few modifications, such as the bearings visualization map
- To make it easier for researchers/ecologists to contribute to the project, given that the vast majority of them are proficient in the R programming language

Golem allowed us to develop the app using self-contained modules, which came in handy as the complexity of the app grew, and more components were added. Specifically, we were able to take each screen presented in chapter 5, and implement it in its own module, mirroring the wizard pattern we chose to follow on the front-end.

Throughout development, we consulted two fantastic books: *Mastering Shiny* [47], and *Engineering Production-Grade Shiny Apps* [48]. The former explains how reactive programming works and provides useful code examples for common tasks. The latter is co-written by the developer of Golem, and it proposes a Waterfall-like methodology for building software in Shiny. It was very helpful for learning to use Golem effectively.

Furthermore, we used the Git version control software to keep track of changes during development. New features were developed in separate branches and merged with the main one when completed.

Vocomatcher is free and open-source.

The source code is available at github.com/jaytohe/ecostats-rshiny.

mic_id	lat	lng
1	14.219238	106.603588
...	...	...

Table 6.1: Example of microphone CSV file accepted by Vocomatcher. "lat" corresponds to latitude, "lng" corresponds to longitude. Each microphone is uniquely identified by "mic_id".

recording_ID	mic_ID	measured_call_datetime	measured_bearing	sex	spectrogram
1	1	2023-01-01 04:31:19	2.20...	M	imgs/LOREM.png
2	3	2023-01-01 04:16:38	5.74...	F	imgs/IPSUM.png
...	...	...	...	...	...

Table 6.2: Example of recordings CSV file supported by Vocomatcher. Two extra columns are required, compared to "ecostats-rshiny" : `sex`, denoting whether the caller is male or female, and `spectrogram` which contains relative file paths to spectrogram images of the calls.

6.2.1 Input Data Format

The columns that Vocomatcher requires in the microphones CSV file are outlined in Table 6.1, while those expected in the recordings CSV file are listed in Table 6.2. This format is very similar to what "ecostats-rshiny" expects (Table 2.4) with the key difference being that two new columns **must** be specified : `sex`, indicating whether the caller is male or female, and `spectrogram`, which provides the relative file paths to the spectrogram images of the recorded calls. These are used to colour-code the arrows on the bearings map based on sex (requirement #14) and to allow the user to compare calls based on spectrograms (requirements #20, #21).

6.3 Challenges Encountered

6.3.1 Displaying Images Inside a DataTable

To create the unmatched calls table shown in Figure 6.1(a), on the left side, *DT* is used, an R/Shiny wrapper of the popular JavaScript *datatables* library. DT handles table pagination, sorting by different columns, filtering, selecting rows, etc.

Only strings can be stored inside a table. However, by passing `escape = FALSE` when initializing the table, strings which contain HTML code are not escaped. For example, if

a cell contains the value "^{Hello World}", this will show up as $_{\text{Hello World}}$ on the front-end because "sup" was interpreted as the subscript HTML tag.

In Shiny, the `imageOutput` function defines an output element for displaying images, while `renderImage` binds to the element and sends the image data.

Given that `imageOutput` returns HTML code, we initially attempted to include it in the datatable with `escape = FALSE`, and employ `renderImage` functions on the server to process and transmit images. Unfortunately, this did not work well because only the images on the current table page, and not on other pages, were being rendered. This is because **the `imageOutput` elements contained in other pages were inaccessible when their corresponding `renderImage` code blocks were being executed.**

We thought of two ways to tackle this issue: a) read-in all the images and embed them in the table *before* the table is rendered, or b) observe which table rows are currently visible on the page and render the relevant spectrogram images on demand.

Both of these approaches required reading, processing and sending the images to the UI without relying on Shiny's built-in tools.

We decided to explore option (b).

6.3.1.1 Solution

DT provides access to the indices of the currently visible rows through an input variable. By monitoring changes to this variable, we can trigger a custom function whenever changes occur. This function iterates over the visible rows, reading each corresponding spectrogram image as raw bytes. These bytes are then converted into a base64 string, which is stored in a key-value data structure, with the row index serving as the key and the base64 string as the value.

Q: Why convert the image to a base64 string?

Displaying images directly on the client-side, which runs in a web browser, typically requires exposing them as files accessible via a URL, e.g. http://example.com/img1.jpeg. To our knowledge, Shiny only supports this method for hardcoded data that ship with the application. Since in our app, the images are user-provided, the most viable solution is to

transcode them into base64, a format universally supported by web browsers.

After processing all the visible rows, the data structure containing the base64-encoded images is sent to the client.

To send custom data to the client without using Shiny's standard input/output elements and render functions, we utilised an advanced feature called `CustomMessageHandler`, as detailed in [48, Chapter 17.4]. This feature allows us to define custom communication behaviour between the R server and the JavaScript client.

On the client side, the key-value data structure is received through the `CustomMessageHandler`. We then loop through the structure, identify the rows by their keys, and update the spectrogram column to display the images as HTML elements with base64 data sources[1].

[1] Relevant GeeksForGeeks article: How to display Base64 images in HTML?

CHAPTER SEVEN

SYSTEM TESTING

This chapter outlines the testing methodology employed for Vocomatcher, including the testing tools utilised. Additionally, it delves into a peculiar bug discovered during testing that affected the ability to remove calls from an existing call group.

7.1 Unit Testing

One way we sought to validate the app's compliance with the gathered software requirements is via unit testing. This method involves isolating individual functions within the source code and verifying whether they work as intended by supplying various values as input. The function's output is then compared against expected results to ensure it behaves as expected.

Specifically, following [47, Chapter 21 "Testing"], we used the `testthat` R package to write some basic test cases for the modules of the app. Unfortunately, due to setbacks during development, we didn't have time to test most components of the app. Nevertheless, the few implemented test cases can be found under "`vocomatcher/tests/testthat`" in the uploaded code zip file on MMS.

7.2 Integration Testing

To assess the interplay between the app's various modules, we performed integration testing after each new feature was implemented. This involved building and executing

the entire application to verify the new component functioned as expected within the existing system. While we recognise the limitations of this manual approach, including the potential for overlooking testing all functionalities each time, the project's time constraints necessitated this strategy. Ideally, a more comprehensive integration testing solution would have been implemented.

7.3 Call Removal Bug

In this section, we discuss an interesting bug we found during integration testing which affected the ability to remove calls from a call group (requirement #17, step #9 in 5.2.3.1).

Specifically, we observed that removing a call z from call group u failed under the following conditions:

1. A call group u which contains call z is created.
2. User clicks "X" to remove call z from group u.
3. Call z is removed.
4. User checks call z and clicks "add call to existing group u"
5. call z is added to group u
6. User clicks "X" to remove call z from group u
7. Call z is **not** removed

Additionally, we identified another scenario where call removal was unsuccessful:

1. User creates call group y containing **only** call h.
2. User clicks "X" to remove call h.
3. Call h is removed, group y is deleted.
4. User creates group t containing **only** call h.

5. User clicks "X" to remove call *h*.

6. Call *h* is **not** removed, group *t* is **not** deleted.

Further investigation revealed that call removal failed whenever an attempt was made to remove a call with the same ID as the most recently removed call. Adding a print statement to the reactive function responsible for removing calls showed that, under the aforementioned scenario, the function was **not** invoked the second time.

Given the absence of apparent issues in our code, we suspected that something in the Shiny framework may be responsible for this behaviour. This led us to recall Shiny's lazy evaluation principle [47, Chapter 3.3.2], which avoids performing calculations until necessary, including **not performing the same operation for a previously computed input**.

To address the issue and ensure that the call removal reactive function ran regardless of whether the same row ID was provided as input, we devised the following solution: Instead of sending only the ID of the call to be removed from the group to the server, **we included the current timestamp from the client system**. This guarantees that the input to the reactive function is different, even if the same ID is to be removed multiple times.

The implemented fix can be found in this commit on the project's GitHub page.

CHAPTER EIGHT

CONCLUSION

This chapter summarises our contributions and reviews whether the project's objectives were achieved and, if so, how they were accomplished. In addition, the limitations of the current version of Vocomatcher are presented, and potential directions for future work on the project are discussed.

8.1 Project Summary

In this dissertation, we thoroughly documented every step of the software engineering methodology we followed to develop Vocomatcher.

Vocomatcher is a visualisation-based web app that assists ecologists in identifying and grouping similar animal vocalizations recorded by multiple microphones during an acoustic survey. It helps pinpoint the individual animals making the sounds and creates a detailed record of which microphones captured each individual, and at what time and location.

This record, called capture history, is exportable into a file format which is compatible with the `ascr` R package used for fitting spatially-explicit capture-recapture models on acoustic data. By fitting these models, several important ecological questions can be answered. For example, the effect of forest canopy density or logging activities on gibbon movement patterns can be studied.

In section 2.2, we reviewed other tools that could be used for identifying vocalising individuals from multiple calls, and in section 2.3 we concluded that there is no existing user-assistive software to match calls via spectrograms and bearings comparison on a user-friendly interface.

As presented in chapter 5 and section 6.1, Vocomatcher offers the following features to expedite the call-matching process:

- An intuitive and easy-to-use interface for matching calls.
- Visualising the study area by plotting the location of each microphone on a geographical map.
- Indicating in said map the estimated angle from which a microphone captured a call by drawing arrows.
- Colour-coding said arrows based on the sex of the vocalising individual.
- Allowing the user to compare the spectrogram images of multiple calls to decide whether they should be matched.
- Exporting the matched calls in a file which is directly readable by the `ascr` R package, as discussed previously.

Vocomatcher is free and open-source.

The source code is available at github.com/jaytohe/ecostats-rshiny.

8.2 Objectives Review

We achieved most of the primary objectives of this project.

The table below illustrates to what extent each primary objective was accomplished and how.

Objective	**Type**	**Fulfilled?**	**Reason/Evidence**
Sketch design mock-ups and revise them based on feedback from the client.	Primary	Yes	See chapter 5.
Develop the new app using a modular approach to improve code debugging and maintainability.	Primary	Yes	As discussed in section 6.2, the app was written in Golem, which allowed us to implement each screen shown in chapter 5 in its own self-contained module.
Integrate useful features from "ecostats-rshiny" into Vocomatcher.	Primary	Mostly	The bearings visualisation map has been ported but other existing features such as resetting the map to its default view by clicking a button or visualising the bearings of matched calls have not been ported yet.

Implement new functionalities as requested by the client, including the ability to compare animal calls based on spectrograms.	Primary	Yes	The most important features have been implemented, including a new interface for matching calls and comparing spectrograms (Figure 6.1).
Properly test the app's components to ensure stability and to minimise the risk of crashing.	Primary	Partly	Basic unit tests were written, but the app has not been properly tested with various inputs, and the integration testing conducted is poor.

8.3 Limitations

Below, we outline some limitations of the current version of the app:

- Requirement #4 has not been implemented yet. Currently, the "Date Select" screen shows the number of calls per day, but the user cannot make a selection.
- Requirement #30 has not been implemented yet. The user can only visualise spectrograms of unmatched calls.
- The user interface does not strictly conform to the wizard UI pattern, allowing users to skip to the next screen without completing all required actions on the current screen. This can potentially lead to errors.
- The app expects no missing values in the input data, which is unrealistic. For example, researchers may not always be able to determine the sex of the calling individual, thereby leaving it blank in the input data.
- The app is not optimized and slows down when processing hundreds of calls, particularly when loading multiple spectrogram images. Moreover, clicking on a row

to calculate the time difference may cause the app to briefly become unresponsive.

8.4 Future Work

First, we plan to conduct comprehensive testing, thoroughly evaluating the business logic and the reactivity of the Shiny app, following the guidelines outlined in [47, Chapter 21 - Testing].

Second, we aim to realize and expand upon the secondary objectives. Our client has expressed interest in developing a location-aware algorithm to suggest likely call groups. We envision this algorithm calculating a similarity index based on numerical features extracted from spectrograms (as in Koe), as well as considering the time difference, sex, and location of the calls.

Finally, we intend to perform benchmarking, as detailed in [47, Chapter 23 - Performance]. This will include profiling the app to identify potential bottlenecks and analysing memory consumption under different scenarios.

REFERENCES

[1] S. R. P.-J. Ross, D. P. O'Connell, J. L. Deichmann, C. Desjonquères, A. Gasc, J. N. Phillips, S. S. Sethi, C. M. Wood, and Z. Burivalova, "Passive acoustic monitoring provides a fresh perspective on fundamental ecological questions," *Functional Ecology*, vol. 37, no. 4, pp. 959–975, 2023. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/1365-2435.14275

[2] L. S. M. Sugai, T. S. F. Silva, J. Ribeiro, José Wagner, and D. Llusia, "Terrestrial Passive Acoustic Monitoring: Review and Perspectives," *BioScience*, vol. 69, no. 1, pp. 15–25, 11 2018. [Online]. Available: https://doi.org/10.1093/biosci/biy147

[3] V. A. Melfi, "Gibbons: probably the most endangered primates in the world," *International Zoo Yearbook*, vol. 46, no. 1, pp. 239–240, 2012. [Online]. Available: https://zslpublications.onlinelibrary.wiley.com/doi/abs/10.1111/j.1748-1090.2012.00177.x

[4] ZSL, "Protecting the world's rarest primate: the Hainan gibbon," May 2024. [Online]. Available: https://www.zsl.org/support-us/back-from-the-brink/protecting-worlds-rarest-primate-hainan-gibbon

[5] O. R. Wearn and P. Glover-Kapfer, "Snap happy: camera traps are an effective sampling tool when compared with alternative methods," *Royal Society Open Science*, vol. 6, no. 3, p. 181748, Mar. 2019. [Online]. Available: https://royalsocietypublishing.org/doi/10.1098/rsos.181748

[6] V. H. Montalvo, C. Sáenz-Bolaños, J. C. Cruz-Díaz, E. Carrillo, and T. K. Fuller, "The use of camera traps and auxiliary satellite telemetry to estimate jaguar population density in northwestern costa rica," *Animals (Basel)*, vol. 12, no. 19, p. 2544, Sep. 2022.

[7] F. Cunha, E. M. dos Santos, R. Barreto, and J. G. Colonna, "Filtering empty camera trap images in embedded systems," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 2438–2446.

[8] P. Glover-Kapfer, C. A. Soto-Navarro, and O. R. Wearn, "Camera-trapping version 3.0: current constraints and future priorities for development," *Remote Sensing in Ecology and Conservation*, vol. 5, no. 3, pp. 209–223, 2019. [Online]. Available: https://zslpublications.onlinelibrary.wiley.com/doi/abs/10.1002/rse2.106

[9] S. Nanaware, R. Shastri, Y. Joshi, and A. Das, "Passive acoustic detection and classification of marine mammal vocalizations," in *2014 International Conference on Communication and Signal Processing*, 2014, pp. 493–497.

[10] R. Gibb, E. Browning, P. Glover-Kapfer, and K. E. Jones, "Emerging opportunities and challenges for passive acoustics in ecological assessment and monitoring," *Methods in Ecology and Evolution*, vol. 10, no. 2, pp. 169–185, 2019. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.13101

[11] "White-cheeked gibbon - akron zoo." [Online]. Available: https://www.akronzoo.org/mammals/white-cheeked-gibbon

[12] A. Gomez Villa, A. Salazar, and F. Vargas, "Towards automatic wild animal monitoring: Identification of animal species in camera-trap images using very deep convolutional neural networks," *Ecological Informatics*, vol. 41, pp. 24–32, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1574954116302047

[13] M. S. Norouzzadeh, A. Nguyen, M. Kosmala, A. Swanson, M. S. Palmer, C. Packer, and J. Clune, "Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning," *Proceedings of the National Academy of Sciences*, vol. 115, no. 25, Jun. 2018. [Online]. Available: https://pnas.org/doi/full/10.1073/pnas.1719367115

[14] M. A. Tabak, M. S. Norouzzadeh, D. W. Wolfson, S. J. Sweeney, K. C. Vercauteren, N. P. Snow, J. M. Halseth, P. A. Di Salvo, J. S. Lewis, M. D. White,

B. Teton, J. C. Beasley, P. E. Schlichting, R. K. Boughton, B. Wight, E. S. Newkirk, J. S. Ivan, E. A. Odell, R. K. Brook, P. M. Lukacs, A. K. Moeller, E. G. Mandeville, J. Clune, and R. S. Miller, "Machine learning to classify animal species in camera trap images: Applications in ecology," *Methods in Ecology and Evolution*, vol. 10, no. 4, pp. 585–590, 2019. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.13120

[15] D. J. Clink and H. Klinck, "Unsupervised acoustic classification of individual gibbon females and the implications for passive acoustic monitoring," *Methods in Ecology and Evolution*, vol. 12, no. 2, pp. 328–341, 2021. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.13520

[16] D. Clink, M. Zafar, A. Ahmad, and A. Lau, "Limited evidence for individual signatures or site-level patterns of variation in male northern gray gibbon (hylobates funereus) duet codas," *International Journal of Primatology*, vol. 42, pp. 1–19, 12 2021.

[17] T. GEISSMANN, "Duet-splitting and the evolution of gibbon songs," *Biological Reviews*, vol. 77, no. 1, pp. 57–76, 2002. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1017/S1464793101005826

[18] D. Borchers and M. Efford, "Spatially explicit maximum likelihood methods for capture-recapture studies," *Biometrics*, vol. 64, pp. 377–85, 07 2008.

[19] T. A. Marques, L. Thomas, S. W. Martin, D. K. Mellinger, J. A. Ward, D. J. Moretti, D. Harris, and P. L. Tyack, "Estimating animal population density using passive acoustics," *Biological Reviews*, vol. 88, no. 2, pp. 287–309, 2013. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1111/brv.12001

[20] R. J. Foster and B. J. Harmsen, "A critique of density estimation from camera-trap data," *The Journal of Wildlife Management*, vol. 76, no. 2, pp. 224–236, 2012. [Online]. Available: https://wildlife.onlinelibrary.wiley.com/doi/abs/10.1002/jwmg.275

[21] M. G. Efford, D. K. Dawson, and D. L. Borchers, "Population density estimated from locations of individuals on a passive detector array," *Ecology*, vol. 90, no. 10,

pp. 2676–2682, 2009. [Online]. Available: https://esajournals.onlinelibrary.wiley.com/doi/abs/10.1890/08-1735.1

[22] K. U. Karanth, "Estimating tiger panthera tigris populations from camera-trap data using capture—recapture models," *Biological Conservation*, vol. 71, no. 3, pp. 333–338, 1995. [Online]. Available: https://www.sciencedirect.com/science/article/pii/000632079400057W

[23] S. C. Silver, L. E. T. Ostro, L. K. Marsh, L. Maffei, A. J. Noss, M. J. Kelly, R. B. Wallace, H. Gómez, and G. Ayala, "The use of camera traps for estimating jaguar panthera onca abundance and density using capture/recapture analysis," *Oryx*, vol. 38, no. 2, p. 148–154, 2004.

[24] M. Trolle and M. Kéry, "Estimation of Ocelot Density in the Pantanal Using Capture-Recapture Analysis of Camera-Trapping Data," *Journal of Mammalogy*, vol. 84, no. 2, pp. 607–614, 05 2003. [Online]. Available: https://doi.org/10.1644/1545-1542(2003)084<0607:EOODIT>2.0.CO;2

[25] Y. Fukuzawa, W. H. Webb, M. D. Pawley, M. M. Roper, S. Marsland, D. H. Brunton, and A. Gilman, "Koe: Web-based software to classify acoustic units and analyse sequence structure in animal vocalizations," *Methods in Ecology and Evolution*, vol. 11, no. 3, pp. 431–441, 2020. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.13336

[26] D. E. KROODSMA and B. E. BYERS, "The Function(s) of Bird Song1," *American Zoologist*, vol. 31, no. 2, pp. 318–328, 08 2015. [Online]. Available: https://doi.org/10.1093/icb/31.2.318

[27] B. Ghojogh, M. Crowley, F. Karray, and A. Ghodsi, *Elements of Dimensionality Reduction and Manifold Learning*. Springer International Publishing, 2023. [Online]. Available: https://books.google.gr/books?id=ScKrEAAAQBAJ

[28] F. Nielsen, *Hierarchical Clustering*. Cham: Springer International Publishing, 2016, pp. 195–211. [Online]. Available: https://doi.org/10.1007/978-3-319-21903-5_8

[29] M. Premoli, D. Baggi, M. Bianchetti, A. Gnutti, M. Bondaschi, A. Mastinu, P. Migliorati, A. Signoroni, R. Leonardi, M. Memo, and S. A. Bonini, "Automatic classification of mice vocalizations using machine learning techniques and convolutional neural networks," *PLOS ONE*, vol. 16, no. 1, pp. 1–16, 01 2021. [Online]. Available: https://doi.org/10.1371/journal.pone.0244636

[30] A. Mielke and K. Zuberbühler, "A method for automated individual, species and call type recognition in free-ranging animals," *Animal Behaviour*, vol. 86, no. 2, pp. 475–482, 2013. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0003347213001978

[31] D. W. Armitage and H. K. Ober, "A comparison of supervised learning techniques in the classification of bat echolocation calls," *Ecological Informatics*, vol. 5, no. 6, pp. 465–473, 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1574954110000919

[32] C. Roemer, J.-F. Julien, P. P. Ahoudji, J.-M. Chassot, M. Genta, R. Colombo, G. Botto, C. A. Negreira, B. A. Djossa, R. K. Ing, A. Hassanin, V. Rufray, Q. Uriot, V.-C. Participants, and Y. Bas, "An automatic classifier of bat sonotypes around the world," *Methods in Ecology and Evolution*, vol. 12, no. 12, pp. 2432–2444, 2021. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.13721

[33] F. Caruso, L. Dong, M. Lin, M. Liu, Z. Gong, W. Xu, G. Alonge, and S. Li, "Monitoring of a nearshore small dolphin species using passive acoustic platforms and supervised machine learning techniques," *Frontiers in Marine Science*, vol. 7, 2020. [Online]. Available: https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2020.00267

[34] C. Hauer, E. Nöth, A. Barnhill, A. Maier, J. Guthunz, H. Hofer, R. X. Cheng, V. Barth, and C. Bergler, "Orca-spy enables killer whale sound source simulation, detection, classification and localization using an integrated deep learning-based segmentation," *Scientific Reports*, vol. 13, no. 1, p. 11106, Jul 2023. [Online]. Available: https://doi.org/10.1038/s41598-023-38132-7

[35] A. N. Allen, M. Harvey, L. Harrell, A. Jansen, K. P. Merkens, C. C. Wall, J. Cattiau, and E. M. Oleson, "A convolutional neural network for automated detection of humpback whale song in a diverse, long-term passive acoustic dataset," *Frontiers in Marine Science*, vol. 8, 2021. [Online]. Available: https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2021.607321

[36] P. Arora, Deepali, and S. Varshney, "Analysis of k-means and k-medoids algorithm for big data," *Procedia Computer Science*, vol. 78, pp. 507–512, 2016, 1st International Conference on Information Security And Privacy 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050916000971

[37] D. Reynolds, *Gaussian Mixture Models*. Boston, MA: Springer US, 2015, pp. 827–832. [Online]. Available: https://doi.org/10.1007/978-1-4899-7488-4_196

[38] B. J. Frey and D. Dueck, "Clustering by passing messages between data points," *Science*, vol. 315, no. 5814, pp. 972–976, 2007. [Online]. Available: https://www.science.org/doi/abs/10.1126/science.1136800

[39] G. Aurélien, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow : Concepts, Tools, and Techniques to Build Intelligent Systems.* O'Reilly Media, 2019, vol. Second edition. [Online]. Available: https://search.ebscohost.com/login.aspx?direct=true&AuthType=sso&db=nlebk&AN=2245240&site=ehost-live&authtype=sso&custid=s3011414

[40] T. O. Kvålseth, "On normalized mutual information: Measure derivations and properties," *Entropy*, vol. 19, no. 11, 2017. [Online]. Available: https://www.mdpi.com/1099-4300/19/11/631

[41] D. Gillespie, D. K. Mellinger, J. Gordon, D. McLaren, P. Redmond, R. McHugh, P. Trinder, X. Deng, and A. Thode, "PAMGUARD: Semiautomated, open source software for real-time acoustic detection and localization of cetaceans." *The Journal of the Acoustical Society of America*, vol. 125, no. 4_Supplement, pp. 2547–2547, 04 2009. [Online]. Available: https://doi.org/10.1121/1.4808713

[42] K. Petersen, C. Wohlin, and D. Baca, "The waterfall model in large-scale development," in *10th International Conference on Product-Focused Software Process Improvement*. Springer, 2009.

[43] H. Beyer and K. Holtzblatt, *Contextual Design: Defining Customer-Centered Systems*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997.

[44] J. Tidwell, *Designing Interfaces: Patterns for Effective Interaction Design*, ser. O'Reilly Series. O'Reilly Media, Incorporated, 2005. [Online]. Available: https://books.google.co.uk/books?id=1D2bAgAAQBAJ

[45] M. Efford, "Data input for secr," Jun. 2024. [Online]. Available: https://cran.r-project.org/web/packages/secr/vignettes/secr-datainput.pdf

[46] B. Stevenson, "Data structure for ascr," Aug. 2019. [Online]. Available: https://github.com/b-steve/ascr/blob/master/vignettes/data-structure.Rnw

[47] H. Wickham, *Mastering Shiny*. O'Reilly Media, 2021. [Online]. Available: https://books.google.co.uk/books?id=CfErEAAAQBAJ

[48] C. Fay, S. Rochette, V. Guyader, and C. Girard, *Engineering Production-Grade Shiny Apps*, ser. Chapman & Hall/CRC The R Series. CRC Press, 2021. [Online]. Available: https://books.google.co.uk/books?id=qExDEAAAQBAJ